American International University Bangladesh

Faculty of Science and Technology

Advanced Database Management System

Project Report on

Sports Tournament Management System

Section C
Group 04

Submitted by

SL	NAME	ID	CONTRIBUTION
1	*MD TANVIR ISLAM SARKER TAMIM*	**23-50786-1**	**25%** Use Case Diagram, Data Insertion, Query basic PL/SQL, Advance PL/SQL
2	*TANVIR MAHATAB ANAN*	**23-50807-1**	**25%** Scenario, ER Diagram, Table Creation, Normalization, Basic Pl/SQL, Schema Diagram, Overall Project verification
3	*JANNATUN NESA JERIN*	**23-50798-1**	**20%** Introduction, Proposal, Class Diagram, Normalization, Conclusion
4	*MD MAHMODUL HASAN*	**23-50794-1**	**15%** Activity diagram and UI planning
5	*SHAHNEWAZ AHMED*	**22-48979-3**	**15%** Documentation and Advance PL/SQL

Contents

Introduction

Many educational institutions, community clubs, corporate companies, and other organizations frequently host sporting events designed to foster teamwork, physical fitness, and the rules of healthy competition. However, behind the scenes, the management of these events is a complex logistical puzzle. Organizers must juggle a multitude of critical activities, including registering participants and teams, verifying eligibility, creating fair match calendars, recording match outcomes, and securely managing historical data about the tournament.

When these administrative procedures are performed manually—relying on endless spreadsheets, paper forms, and fragmented text or email chains—the process quickly becomes overwhelming. Manual management takes an immense amount of time and is highly vulnerable to human error. Schedulers often face the daunting task of avoiding time and venue conflicts, while lost paperwork, misplaced scorecards, and communication bottlenecks can lead to frustrated participants and delayed matches. Ultimately, the administrative burden detracts from the actual purpose of the event.

The emergence of the Sports Tournament Management System (STMS) provides a comprehensive solution to these logistical nightmares. By transitioning to a centralized digital platform, organizers can organize, process, and prepare tournament data seamlessly, making the entire lifecycle of tournament management significantly easier.

Implementing a robust system transforms chaos into a streamlined operation by offering several key advantages:

- Automated Registration: Players and teams can register, and eliminating manual data entry.
- Dynamic Scheduling & Bracket Generation: The system can automatically generate match calendars, assign venues, and build tournament brackets while instantly flagging any scheduling conflicts.
- Real-Time Scoring & Updates: Referees or officials can input scores directly from the field using mobile devices. The system then automatically updates leaderboards, calculates standings, and notifies advancing teams.

By automating these time-consuming administrative tasks, a Sports Tournament Management System ensures a flawless execution of the event. It allows administrators, coaches, and athletes to focus less on paperwork and logistics, and more on what truly matters: the spirit of the game.

Project Proposal

The **Sports Tournament Management System** is a database-driven application developed to simplify and automate the management of sports tournaments. It provides a centralized platform for storing, organizing, and managing tournament-related information efficiently. The system is designed to overcome the limitations of manual tournament management, such as data inconsistency, scheduling difficulties, delayed result updates, and record maintenance issues.

Objectives

The proposed system aims to achieve the following objectives:

- Centralize all tournament-related information within a single database.
- Simplify tournament organization through digital management.
- Reduce manual work and minimize human errors.
- Ensure accurate, consistent, and secure data management.
- Provide quick and easy access to tournament information for authorized users.

Key Features

The proposed **Sports Tournament Management System** includes the following key features:

- **User and Role Management:** Supports four user roles—System Administrator, Tournament Organizer, Team Captain, and Viewer. Each user is assigned a specific role with defined responsibilities and permissions.
- **Tournament Management:** Enables Tournament Organizers to create and manage tournaments by maintaining tournament details, schedules, and related information.
- **Team and Player Management:** Allows Team Captains to manage a single team and maintain player information, while ensuring that each player belongs to only one team.
- **Tournament Registration:** Allows teams to register for multiple tournaments and enables each tournament to accommodate multiple participating teams.
- **Match Management:** Supports the scheduling and management of matches, including match date, time, venue, status, and the participating teams with their respective roles (Home Team and Away Team).
- **Result Management:** Records and maintains match results, including team scores and match status, allowing tournament outcomes to be tracked efficiently.
- **Result Viewing:** Enables viewers to access published match results and follow tournament progress without modifying any tournament data.

- **Centralized Database:** Stores all tournament-related information in a structured relational database, ensuring data consistency, minimizing redundancy, and supporting efficient data retrieval.

Database Management

The system stores essential information, including users, tournaments, teams, players, matches, and results, in a structured relational database. Well-defined relationships among these entities help maintain data integrity, minimize redundancy, and support efficient data retrieval and management.

Expected Benefits

The proposed system offers several benefits, including:

- Improved efficiency in tournament management.
- Reduced administrative workload.
- Faster access to tournament information.
- Better communication among organizers, participants, and viewers.
- Accurate and reliable record management.
- Improve data consistency an reduce redundancy.

Class Diagram

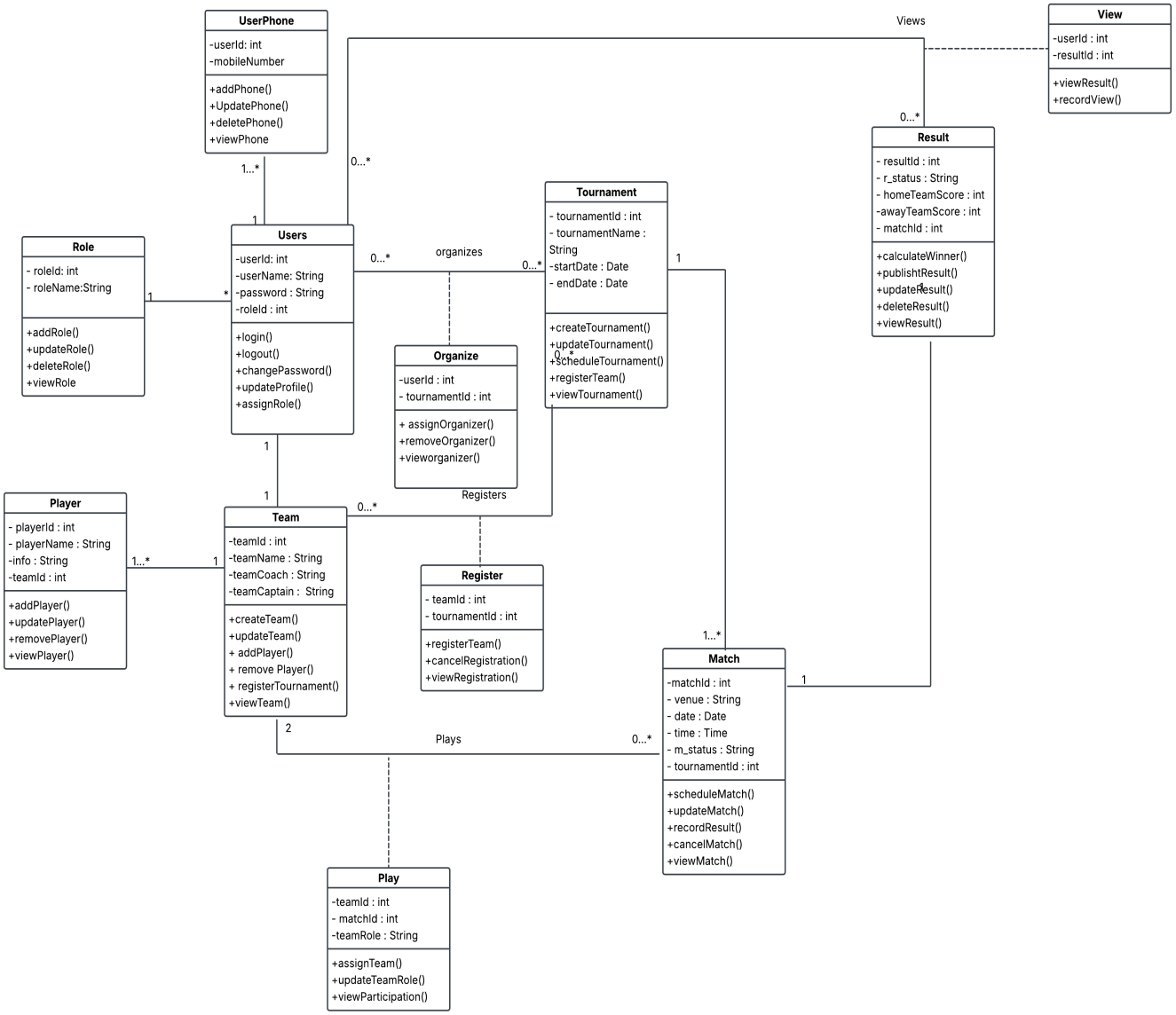


Fig: Class Diagram (Sports Tournament Management System)

Use Case Diagram

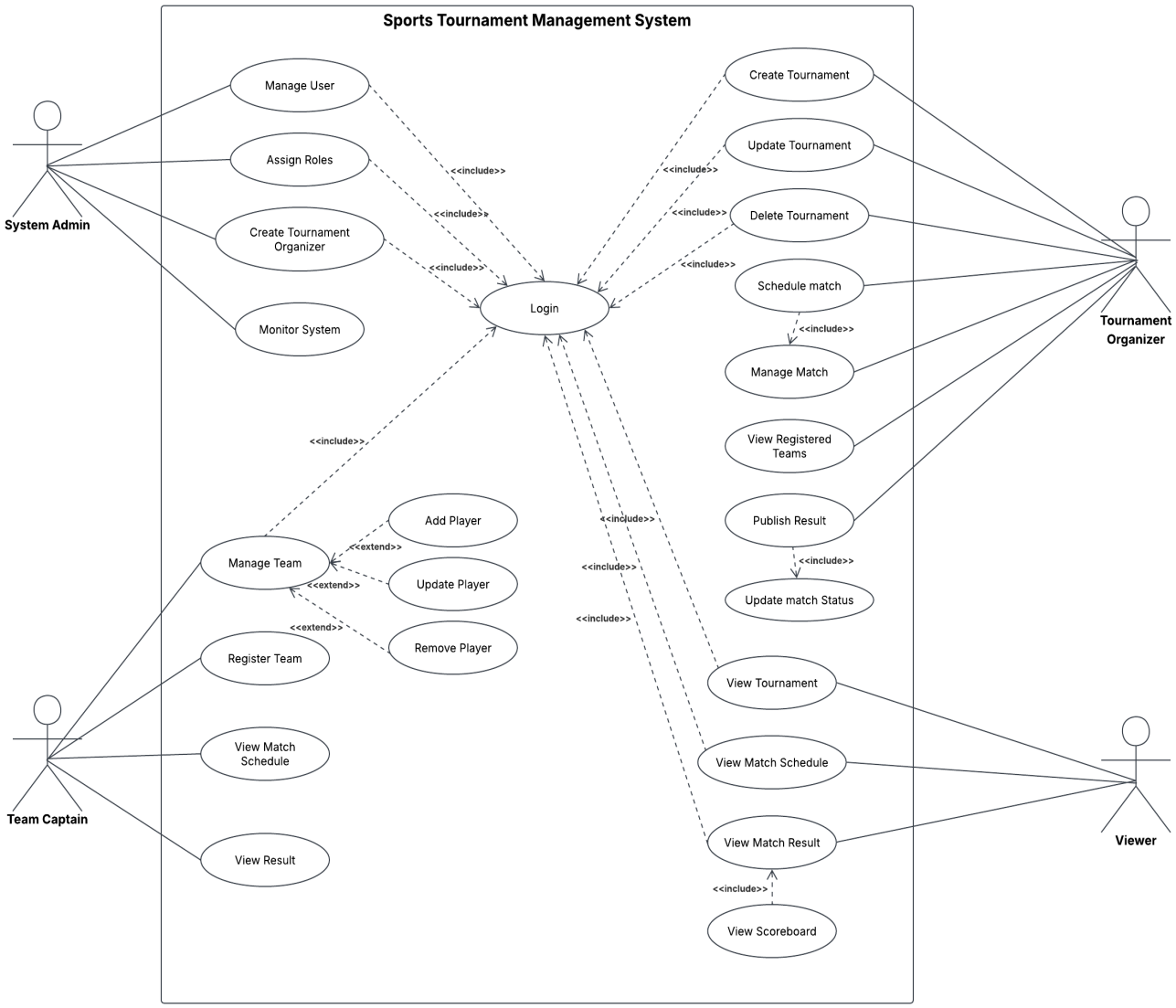


Fig: Use Case Diagram (Sports Tournament Management System)

Activity Diagram

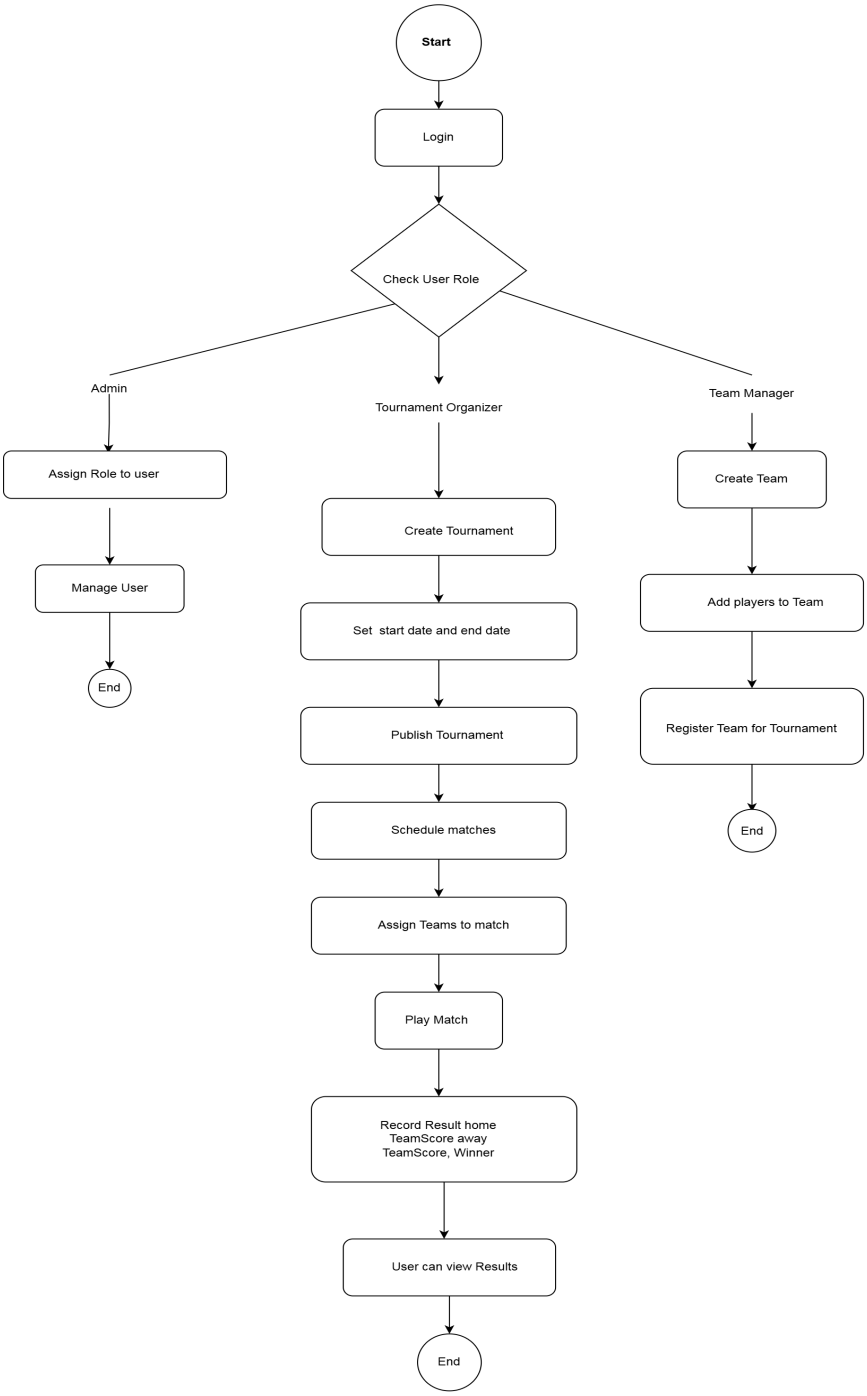
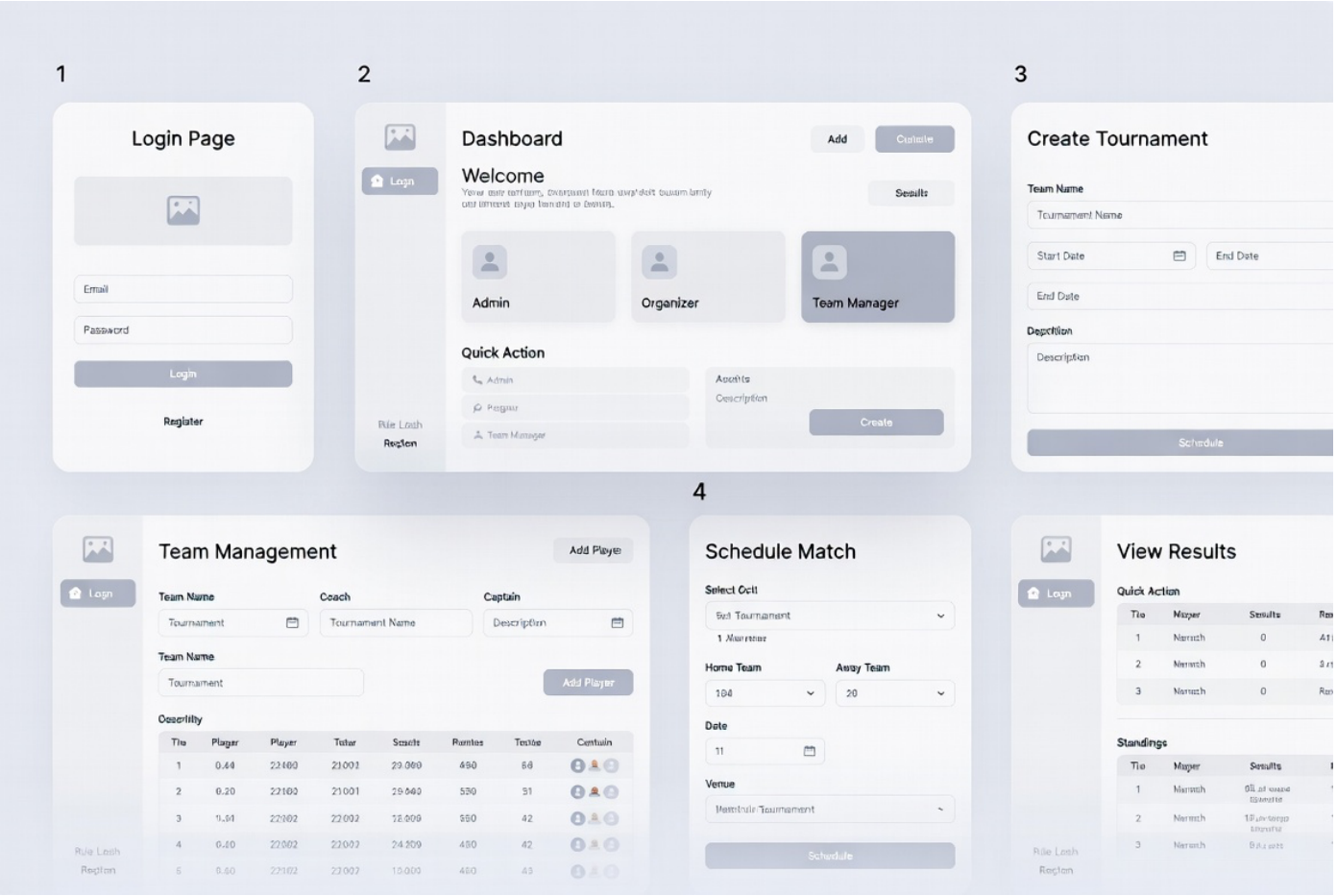
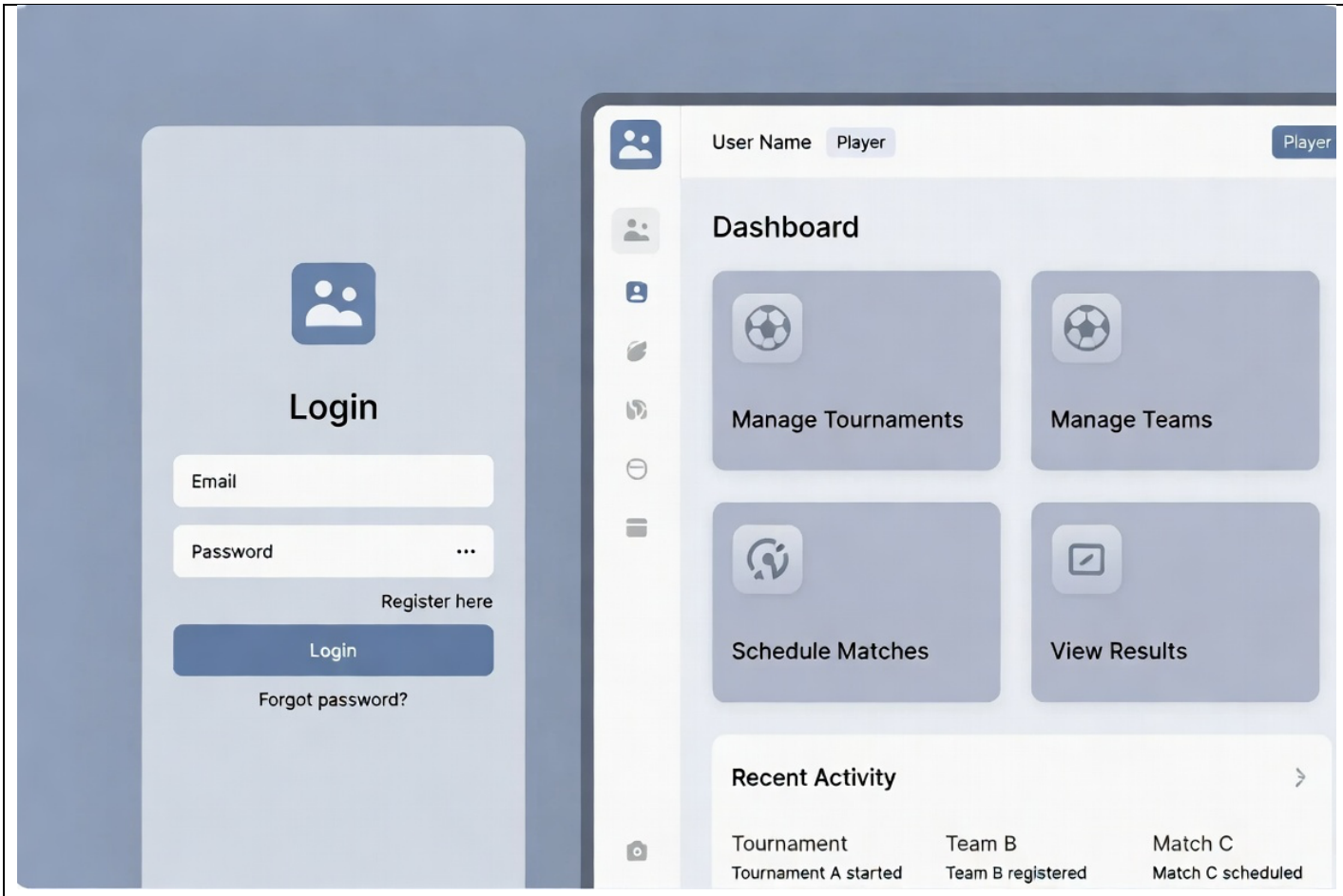
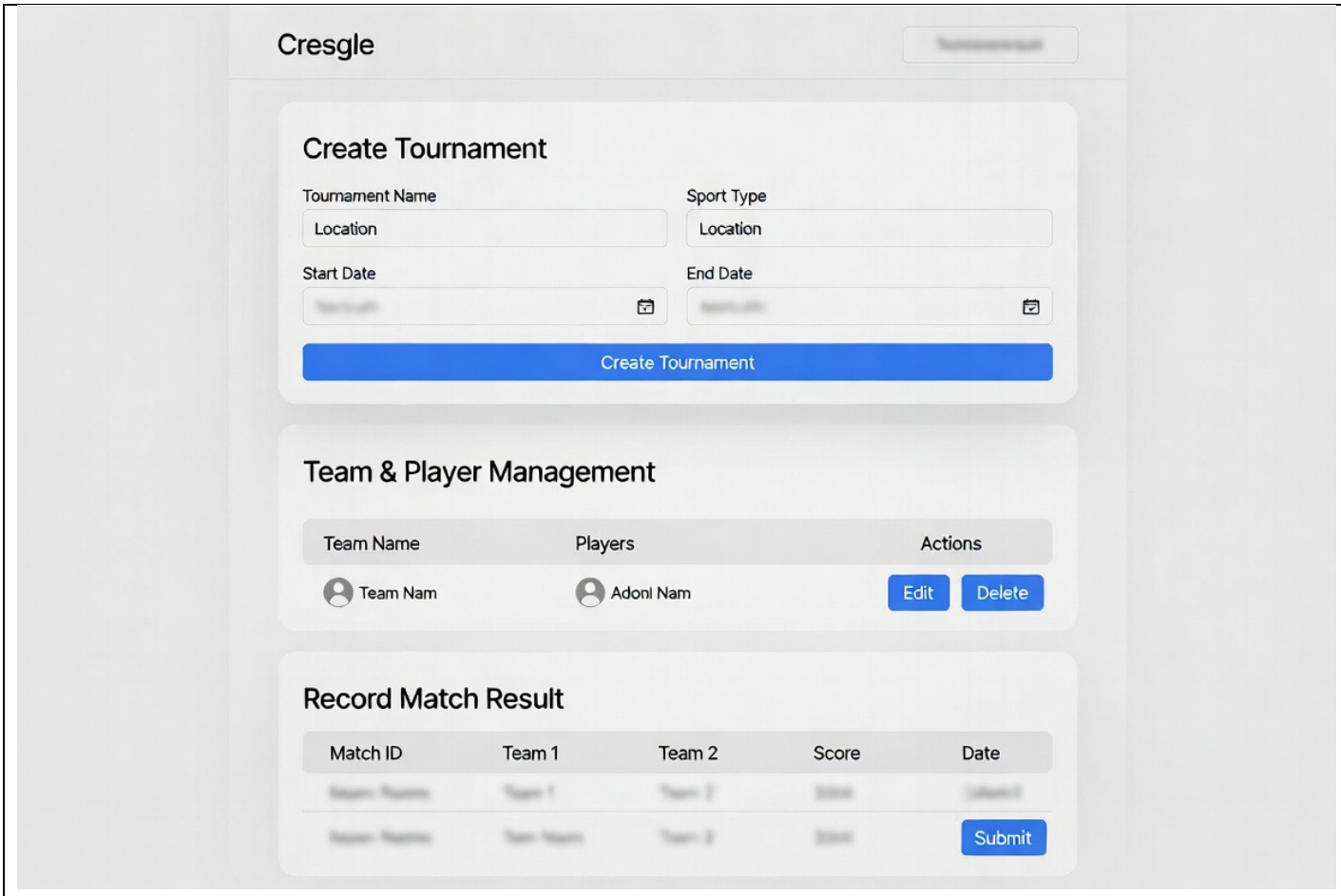


Fig: Activity Diagram (Sports Tournament Management System)

User Interface Planning




Login
Email
Password
Register here
Login
Forgot password?
User Name
Player
Player
Dashboard
Manage Tournaments
Manage Teams
Schedule Matches
View Results
Recent Activity
Tournament
Tournament A started
Team B
Team B registered
Match C
Match C scheduled


Cresgle
Create Tournament
Tournament Name
Location
Sport Type
Location
Start Date
End Date
Create Tournament
Team & Player Management
Team Name
Players
Actions
Team Nam
Adonl Nam
Edit
Delete
Record Match Result
Match ID
Team 1
Team 2
Score
Date
Submit

Scenario Description

Sports Tournament Management System

Project Scenario

In a sports tournament management system, all individuals are stored as users, identified by a user ID, name, mobile no, and password. The user may have more than one mobile number. A user is assigned exactly one role, but a role may be assigned to many users, such as System Admin, Tournament Organizer, Team Captain, and Viewer. A system administrator can create multiple tournament organizers in order to create a hierarchy, but only one admin can create an organizer. Based on their role, a user acting as an organizer organize many tournaments also a tournament is managed by many organizers. A tournament is identified by a tournament ID, name, start date, and end date. A user acting as a team captain manages exactly one team, and a team is managed by exactly one captain. A team is identified by a team ID, team Coach name, team Captain and team name. A team contains many players, and a player belongs to exactly one team and each player is identified by a player ID, player name, and info. A team may register for many tournaments, and a tournament can have many registered teams. A tournament has many matches, but a match belongs to exactly one tournament. A match is identified by a match ID, Date, time, Venue, and status. A match is played by exactly two teams. While playing a match the team role is stored. A match holds exactly one result, and a result belongs to exactly one match. Result has result ID, winner, Home Team Score and Away Team Score. Winner, can be derived from Home team Score and Away Team Score. Finally, a user acting as a viewer can view many results, and a result can be viewed by many viewers.

ER Diagram

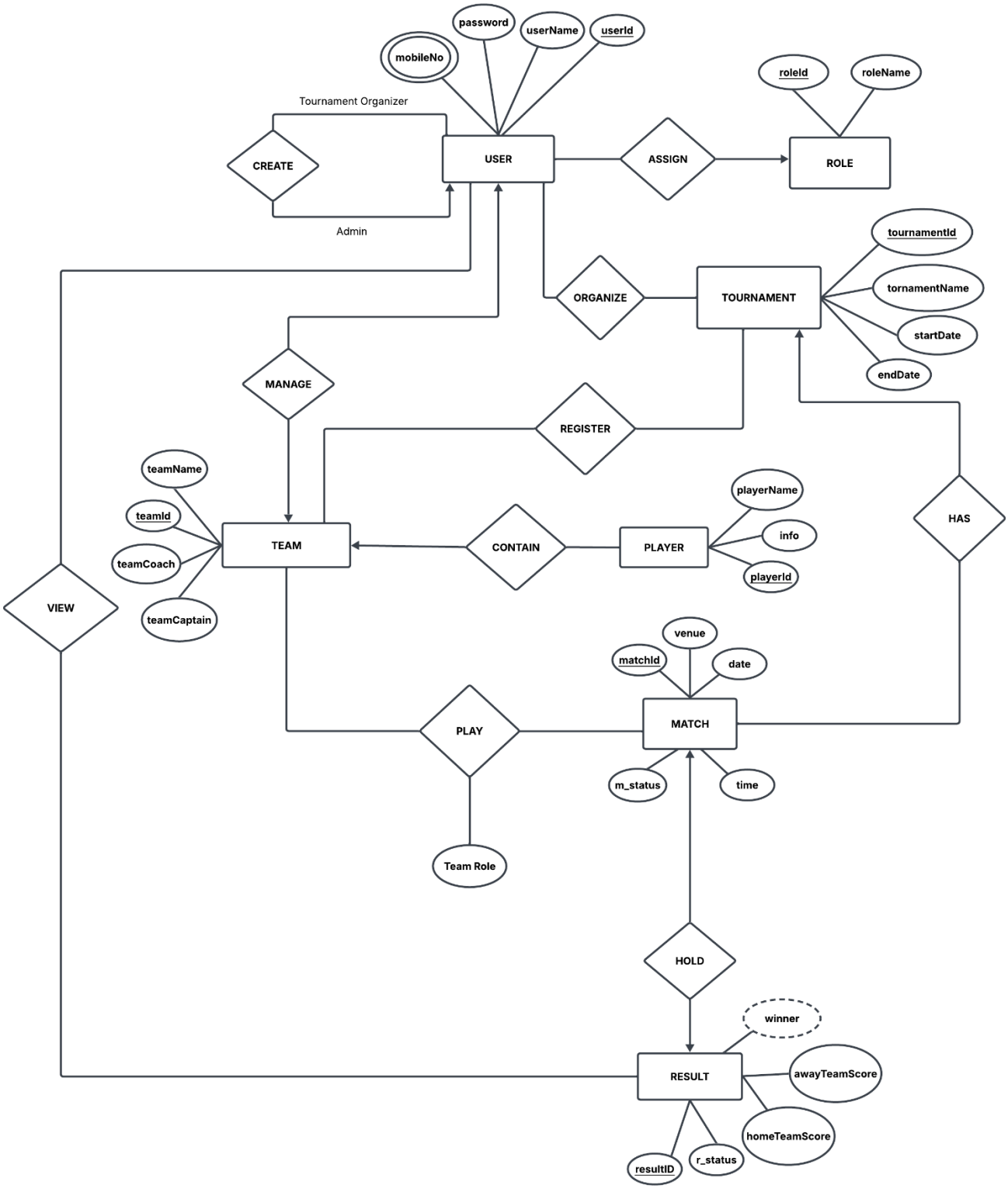


Fig: ER Diagram (Sports Tournament Management System)

Normalization

Assign Relationship:

UNF:
Assign (userId, userName, mobileNo, password, roleId, roleName)

1NF:
mobileNo is a multi-valued attribute

1. userId, userName, mobileNo, password, roleId, roleName

2NF:

1. userId, userName, mobileNo, password
2. roleId, roleName

3NF:
There is no transitive dependency

1. roleId, roleName
2. userId, userName,password,mobileNo

Table creation:

1. roleId, roleName
2. userId, userName, password, **roleId**
3. userId, mobileNo

Create Relationship:

UNF:
Create (userId, userName, mobileNo, password)

1NF:
mobileNo is a multi-valued attribute.

1. userId, userName, mobileNo, password

2NF:
There is partial dependency. Relation already in 2NF.

1. userId, userName, mobileNo, password

3NF:
There is no transitive dependency. Relation already in 3NF.

1. userId, userName, mobileNo, password

Table creation:

1. userId, userName, password
2. userId, mobileNo

Organize Relationship:

UNF:
Organize (userId, userName, mobileNo, password, tournamentId, tournamentName, startDate, endDate)

1NF:
mobileNo is a multi-valued attribute.
1. userId, userName, mobileNo, password, tournamentId, tournamentName, startDate, endDate

2NF:
1. userId, userName, mobileNo, password
2. tournamentId, tournamentName, startDate, endDate

3NF:
There is no transitive dependency. Relation already in 3NF.
1. userId, userName, mobileNo, password
2. tournamentId, tournamentName, startDate, endDate

Table creation:
1. userId, userName, password
2. userId, mobileNo
3. tournamentId, tournamentName, startDate, endDate
4. **userId**, **tournamentId**

Register relationship:

UNF:
Register (teamId, teamName, teamCoach, teamCaptain, tournamentId, tournamentName, startDate, endDate)

1NF:
There is no multi-valued attribute. Relation already in 1NF.
1. teamId, teamName, teamCoach, teamCaptain, tournamentId, tournamentName, startDate, endDate

2NF:
1. teamId, teamName, teamCoach, teamCaptain
2. tournamentId, tournamentName, startDate, endDate

3NF:
There is no transitive dependency. Relation already in 3NF.
1. teamId, teamName, teamCoach, teamCaptain
2. tournamentId, tournamentName, startDate, endDate

Table creation:
1. teamId, teamName, teamCoach, teamCaptain
2. tournamentId, tournamentName, startDate, endDate
3. **teamId**, **tournamentId**

Manage relationship:

UNF:
Manage (userId, userName, mobileNo, password, teamId, teamName, teamCoach, teamCaptain)

1NF:
mobileNo is a multi-valued attribute.

1. userId, userName, mobileNo, password, teamId, teamName, teamCoach, teamCaptain

2NF:

1. userId, userName, mobileNo, password
2. teamId, teamName, teamCoach, teamCaptain

3NF:
There is no transitive dependency. Relation already in 3NF.

1. userId, userName, mobileNo, password
2. teamId, teamName, teamCoach, teamCaptain

Table creation:

1. userId, userName, password,
2. userId, mobileNo
3. teamId, teamName, teamCoach, teamCaptain, **userId**

Contain Relationship:

UNF:
contain (teamId, teamName, teamCoach, teamCaptain, playerId, playerName, info)

1NF:
There is no multi-valued attribute. Relation already in 1NF.

1. teamId, teamName, teamCoach, teamCaptain, playerId, playerName, info

2NF:
There is partial dependency

1. teamId, teamName, teamCoach, teamCaptain
2. playerId, playerName, info

3NF:
There is no transitive dependency. Relation already in 3NF.

1. teamId, teamName, teamCoach, teamCaptain
2. playerId, playerName, info

Table creation:

1. teamId, teamName, teamCoach, teamCaptain
2. playerId, playerName, info, **teamId**

Has Relationship:

UNF:
Has (tournamentId, tournamentName, startDate, endDate, matchId, venue, date, time, m_status)

1NF:
There is no multi-valued attribute. Relation already in 1NF.

1. tournamentId, tournamentName, startDate, endDate, matchId, venue, date, time, m_status)

2NF:

1. tournamentId, tournamentName, startDate, endDate
2. matchId, venue, date, time, m_status

3NF:
There is no transitive dependency. Relation already in 3NF.

1. tournamentId, tournamentName, startDate, endDate
2. matchId, venue, date, time, m_status

Table creation:

1. tournamentId, tournamentName, startDate, endDate
2. matchId, venue, date, time, m_status, **tournamentId**

Play Relationship:

UNF:
Play(teamId, teamName, teamCoach, teamCaptain, matchId, venue, date, time, m_status,teamRole)

1NF:
There is no multi-valued attribute. Relation already in 1NF.

1. teamId, teamName, teamCoach, teamCaptain,matchId, venue, date, time, m_status, teamRole

2NF:

1. teamId, teamName, teamCoach, teamCaptain
2. matchId, venue, date, time
3. teamRole

3NF:

1. teamId, teamName, teamCoach, teamCaptain
2. matchId, venue, date, time, m_status,
3. teamRole

Table creation:

1. teamId, teamName, teamCoach, teamCaptain
2. matchId, venue, date, time, m_status
3. **teamId**,**matchId** ,teamRole

Hold Relationship:

UNF:
Hold (matchId, venue, date, time, m_status, resultId, r_status, homeTeamScore, awayTeamScore, winner)

1NF:
There is no multi-valued attribute. Relation already in 1NF.

1. matchId, venue, date, time, m_status, resultId, r_status, homeTeamScore, awayTeamScore, winner

2NF:

1. matchId, venue, date, time, m_status,
2. resultId, r_status, homeTeamScore, awayTeamScore,winner

3NF:
There is no transitive dependency. Relation already in 3NF.

1. matchId, venue, date, time, m_status
2. resultId, r_status, homeTeamScore, awayTeamScore

Table creation:

1. matchId, venue, date, time, m_status,
2. resultId, r_status, homeTeamScore, awayTeamScore, **matchId**

View Relationship:
UNF:
View (userId, userName, mobileNo, password, resultId, r_status, homeTeamScore, awayTeamScore)

1NF:
mobileNo is a multi-valued attribute.

1. userId, userName, mobileNo, password, resultId, r_status, homeTeamScore, awayTeamScore,winner

2NF:

1. userId, userName, mobileNo, password,
2. resultId, r_status, homeTeamScore, awayTeamScore, winner

3NF:
There is no transitive dependency. Relation already in 3NF.

1. userId, userName, mobileNo, password,
2. resultId, r_status, homeTeamScore, awayTeamScore

Table creation:

1. userId, userName, password
2. userId, mobileNo
3. resultId, r_status, homeTeamScore, awayTeamScore
4. **userId**, **resultId**

Temporary Table:

1. roleId, roleName

2. userId, userName, password, **roleId**
3. userId, mobileNo
4. ~~userId, userName, password~~
5. ~~userId, mobileNo~~
6. ~~userId, userName, password~~
7. ~~userId, mobileNo~~
8. tournamentId, tournamentName, startDate, endDate
9. **userId**, **tournamentId**
10. ~~teamId, teamName, teamCoach, teamCaptain~~
11. ~~tournamentId, tournamentName, startDate, endDate~~
12. **teamId**, **tournamentId**
13. ~~userId, userName, password,~~
14. ~~userId, mobileNo~~
15. teamId, teamName, teamCoach, teamCaptain, **userId**
16. ~~teamId, teamName, teamCoach, teamCaptain~~
17. playerId, playerName, info, **teamId**
18. ~~tournamentId, tournamentName, startDate, endDate~~
19. matchId, venue, date, time, m_status, homeTeamId , awayTeamId, **tournamentId**
20. ~~teamId, teamName, teamCoach, teamCaptain~~
21. ~~matchId, venue, date, time, m_status~~
22. **teamId**,**matchId** ,teamRole
23. ~~matchId, venue, date, time, m_status,~~
24. resultId, r_status, homeTeamScore, awayTeamScore, **matchId**
25. ~~userId, userName, password~~
26. ~~userId, mobileNo~~
27. ~~resultId, r_status, homeTeamScore, awayTeamScore~~
28. **userId**, **resultId**

Final Table:

1. roleId, roleName
2. userId, userName, password, **roleId**
3. userId, mobileNo
4. tournamentId, tournamentName, startDate, endDate
5. **userId**, **tournamentId**
6. **teamId**, **tournamentId**
7. teamId, teamName, teamCoach, teamCaptain, **userId**
8. playerId, playerName, info, **teamId**
9. matchId, venue, date, time, m_status, **tournamentId**
10. **teamId**,**matchId** ,teamRole
11. resultId, r_status, homeTeamScore, awayTeamScore, **matchId**
12. **userId**, **resultId**

Final Table with Table Name:

1. **Role (**roleId, roleName)
2. **Users (**userId, userName, password, roleId)
3. **UserPhone** (userId, mobileNo)
4. **Tournament** (tournamentId, tournamentName, startDate, endDate)
5. **Team** (teamId, teamName, teamCoach, teamCaptain, **userId**)
6. **Player** (playerId, playerName, info, **teamId**)
7. **Match** (matchId, venue, date, time, m_status, **tournamentId**)
8. **Result** (resultId, r_status, homeTeamScore, awayTeamScore, **matchId**)
9. **Organize (userId, tournamentId)**
10. **Register (teamId, tournamentId)**
11. **Play (teamId,matchId** ,teamRole)
12. **Views (userId, resultId)**

Schema Diagram

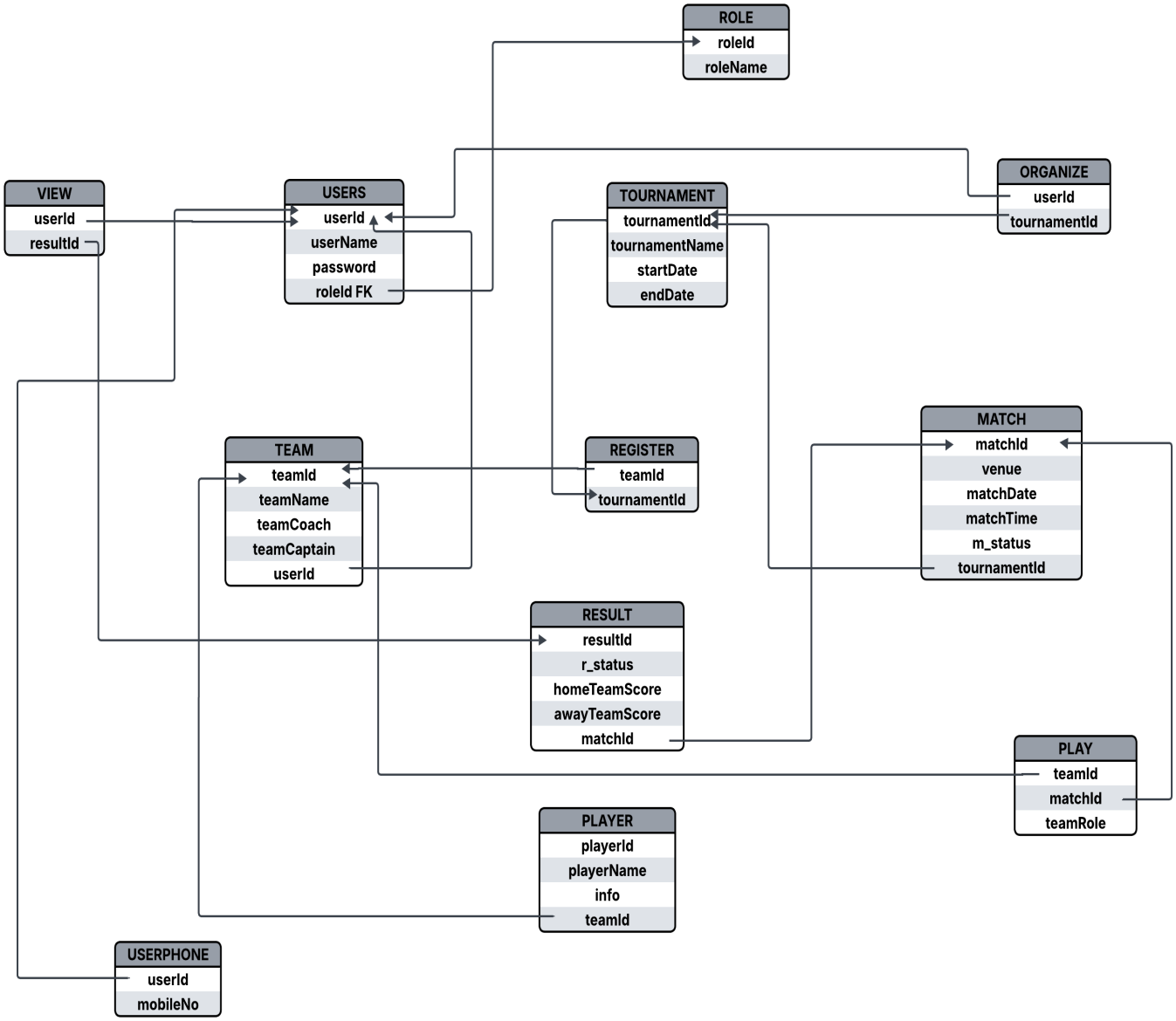
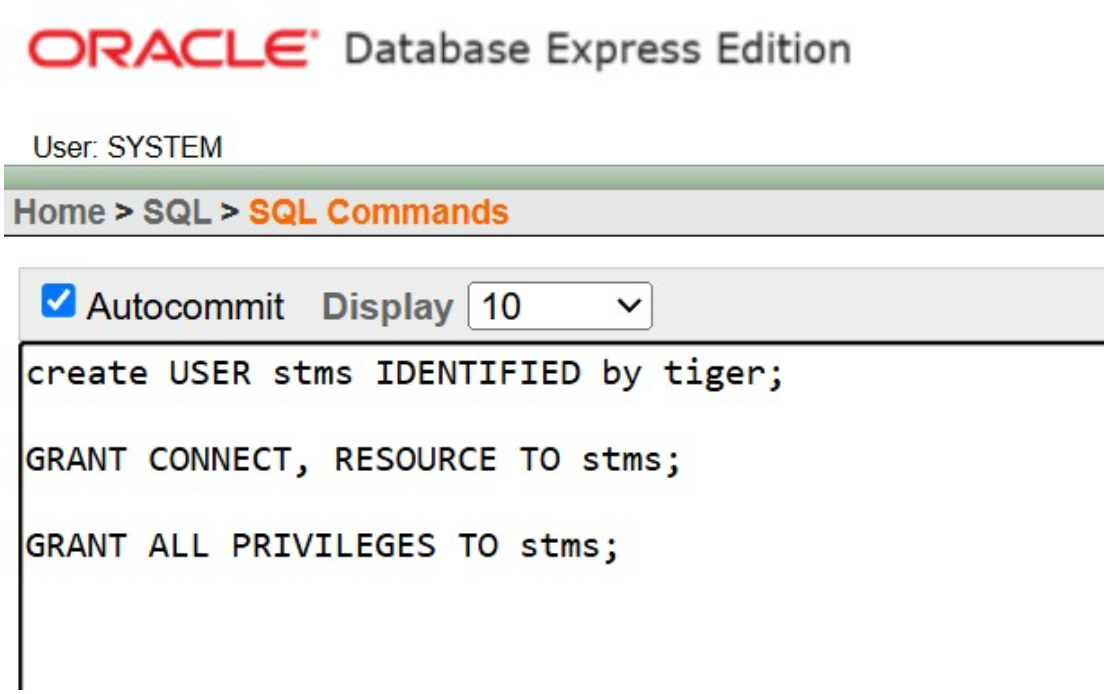


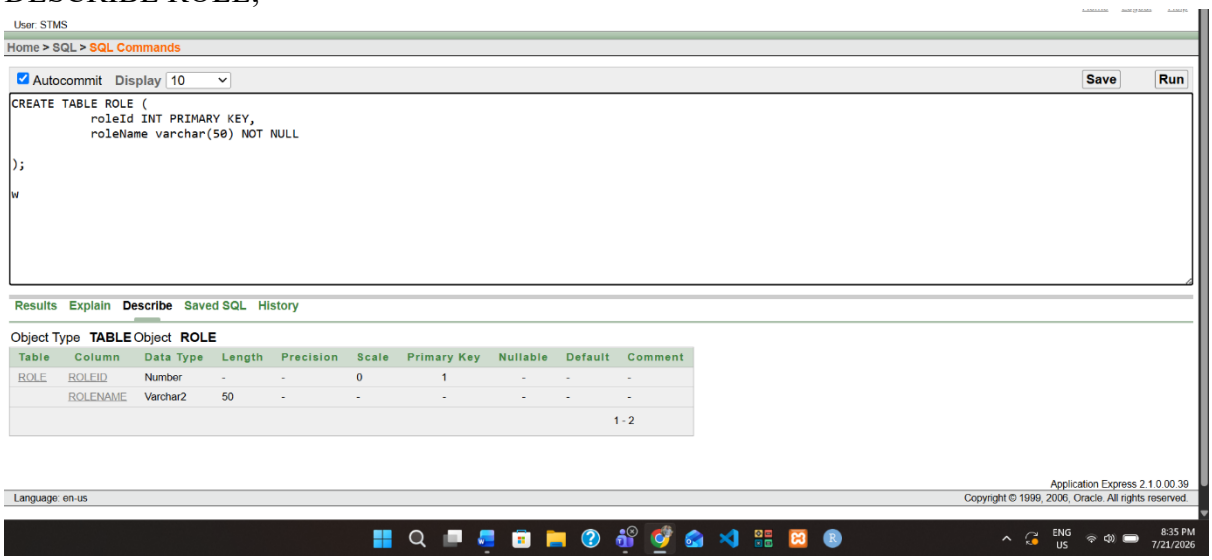
Table Creation Using SQL

User Creation:

Create USER stms IDENTIFIED by tiger;
GRANT CONNECT, RESOURCES TO stms;
GRANT ALL PRIVILEGES TO stms;



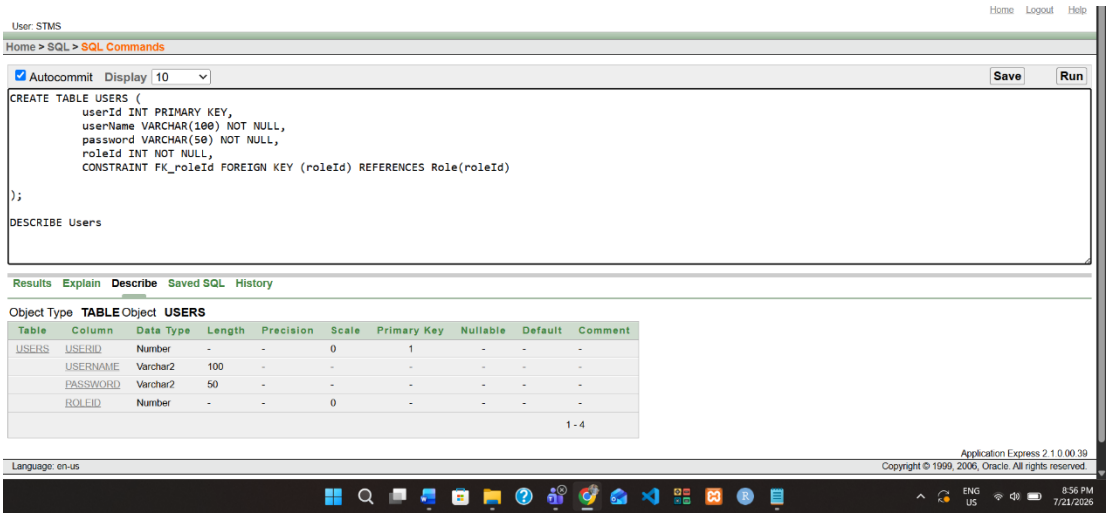
1. CREATE TABLE Role (roleId INT PRIMARY KEY, roleName VARCHAR (50) NOT NULL);
   DESCRIBE ROLE;



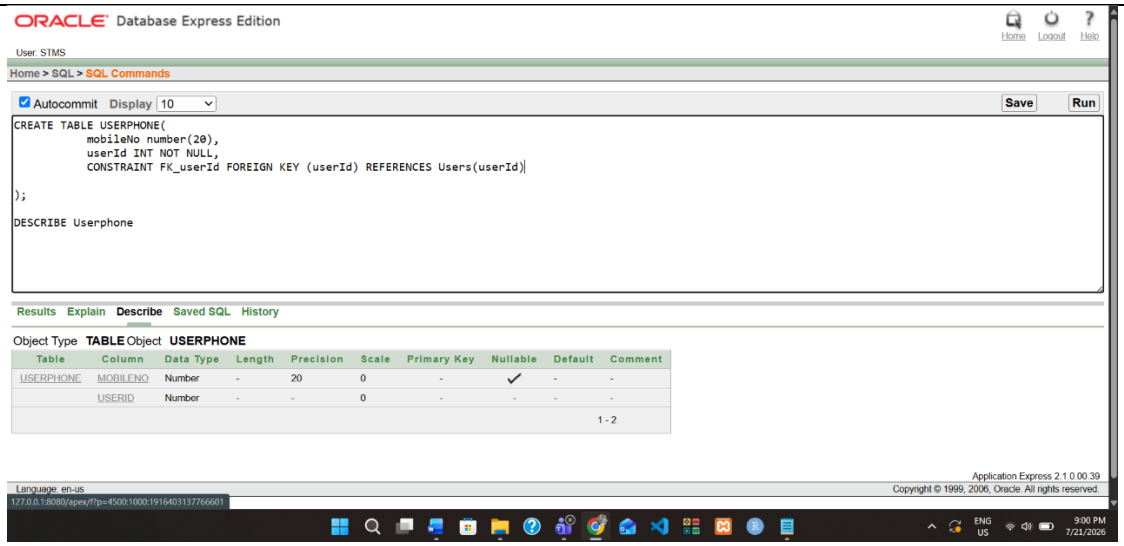
2. CREATE TABLE USERS (
   userId INT PRIMARY KEY,
   userName VARCHAR(100) NOT NULL,
   password VARCHAR(50) NOT NULL,
   roleId INT NOT NULL,
   CONSTRAINT FK_roleId FOREIGN KEY (roleId) REFERENCES Role(roleId)

   );

   DESCRIBE Users

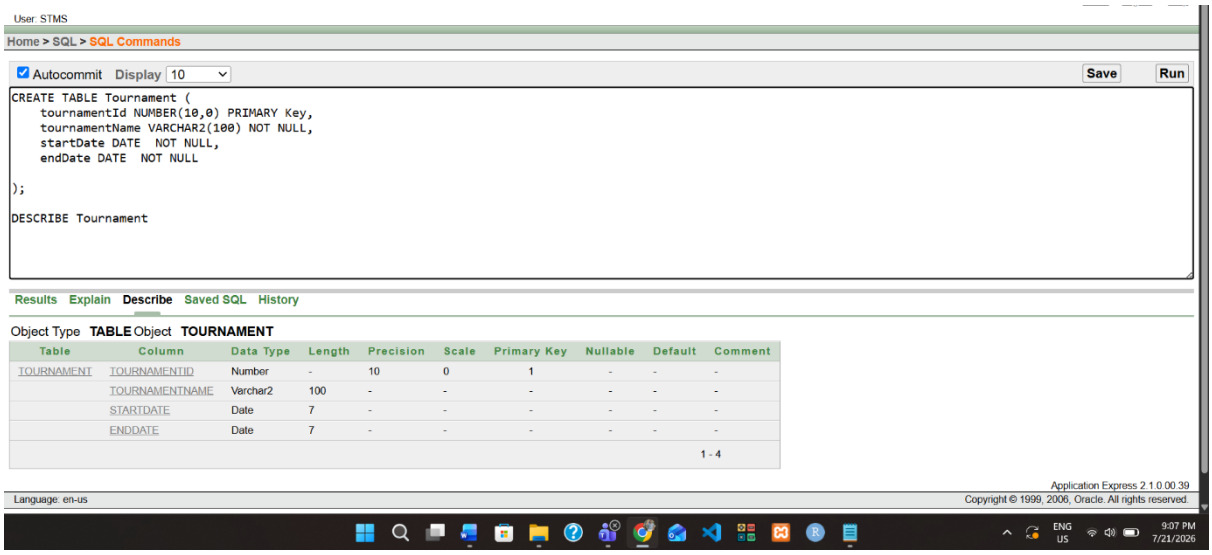


3. CREATE TABLE USERPHONE(
   mobileNo number(20),
   userId INT NOT NULL,
   CONSTRAINT FK_userId FOREIGN KEY (userId) REFERENCES Users(userId));
   DESCRIBE Userphone



4. CREATE TABLE Tournament (
   tournamentId NUMBER(10,0) PRIMARY Key,
   tournamentName VARCHAR2(100) NOT NULL,
   startDate DATE NOT NULL,
   endDate DATE NOT NULL

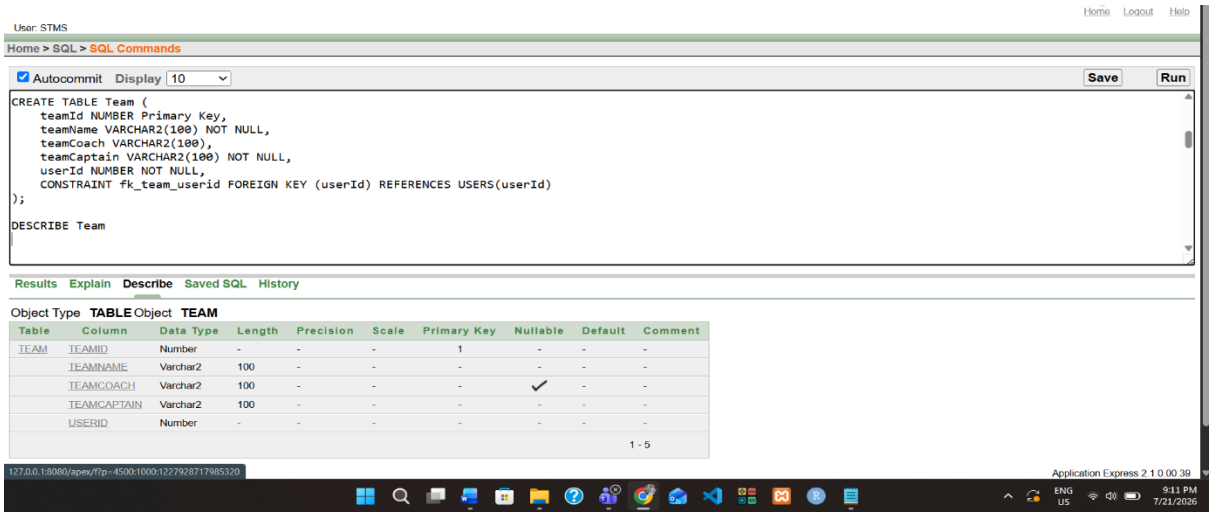
   );
   DESCRIBE Tournament



5. CREATE TABLE Team (
   teamId NUMBER Primary Key,
   teamName VARCHAR2(100) NOT NULL,
   teamCoach VARCHAR2(100),
   teamCaptain VARCHAR2(100) NOT NULL,
   userId NUMBER NOT NULL,

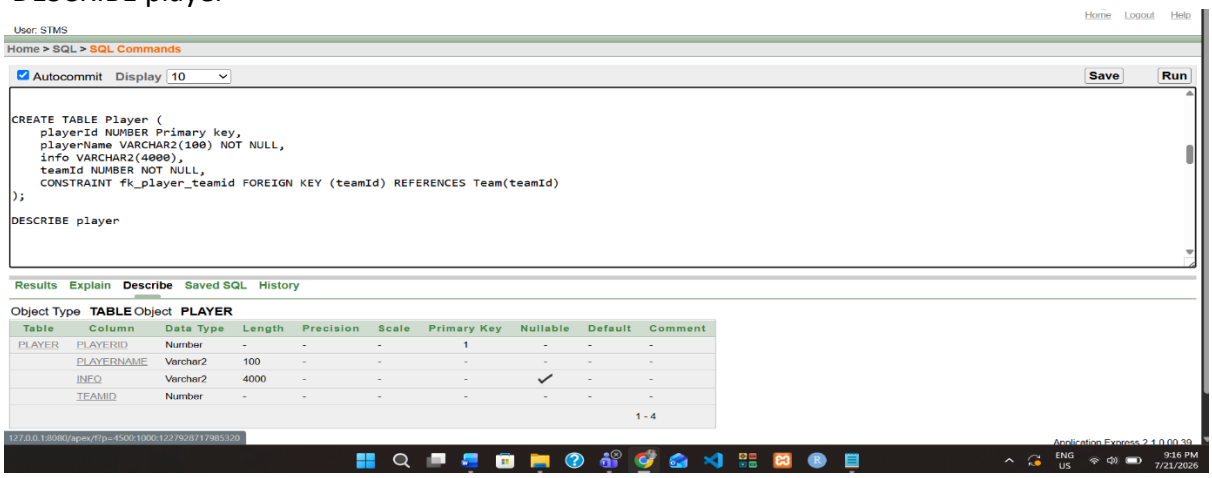
CONSTRAINT fk_team_userid FOREIGN KEY (userId) REFERENCES USERS(userId)
);

DESCRIBE Team



6. CREATE TABLE Player (
playerId NUMBER Primary key,
playerName VARCHAR2(100) NOT NULL,
info VARCHAR2(4000),
teamId NUMBER NOT NULL,
CONSTRAINT fk_player_teamid FOREIGN KEY (teamId) REFERENCES Team(teamId)
);

DESCRIBE player

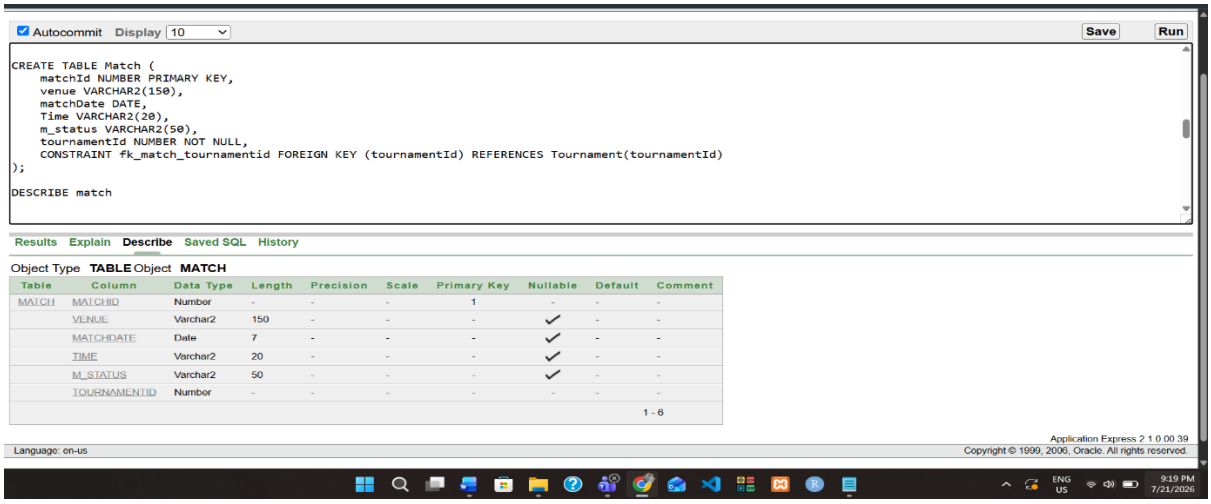


Fig

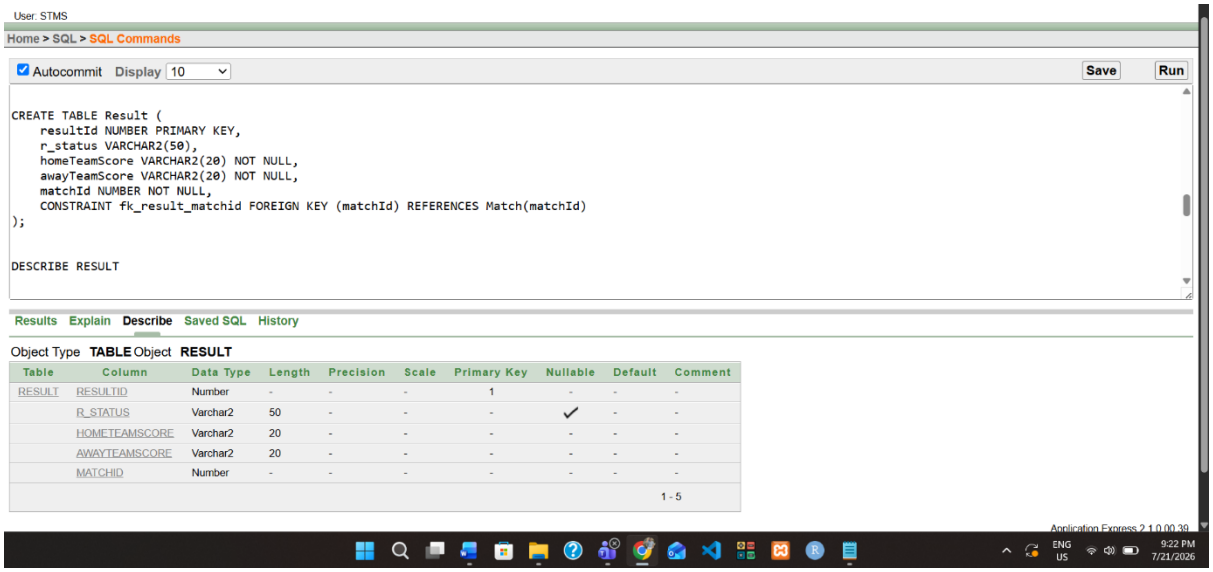
7. CREATE TABLE Match (
matchId NUMBER PRIMARY KEY,
venue VARCHAR2(150),
matchDate DATE,
Time VARCHAR2(20),
m_status VARCHAR2(50),
tournamentId NUMBER NOT NULL,

CONSTRAINT fk_match_tournamentid FOREIGN KEY (tournamentId) REFERENCES Tournament(tournamentId)
);

DESCRIBE match



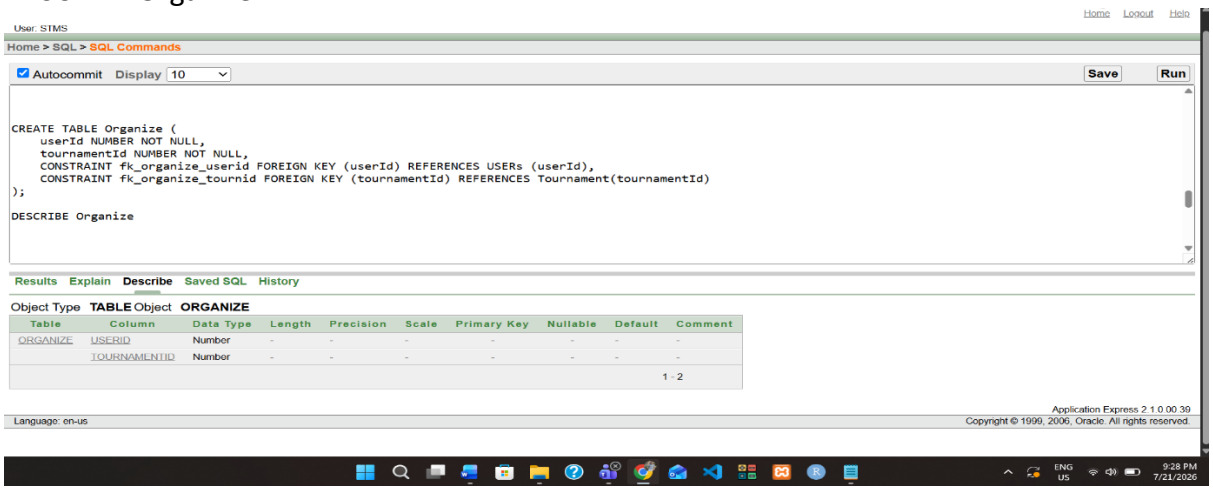
8. CREATE TABLE Result (
   resultId NUMBER PRIMARY KEY,
   r_status VARCHAR2(50),
   homeTeamScore VARCHAR2(20) NOT NULL,
   awayTeamScore VARCHAR2(20) NOT NULL,
   matchId NUMBER NOT NULL,
   CONSTRAINT fk_result_matchid FOREIGN KEY (matchId) REFERENCES Match(matchId)
   );
   DESCRIBE RESULT



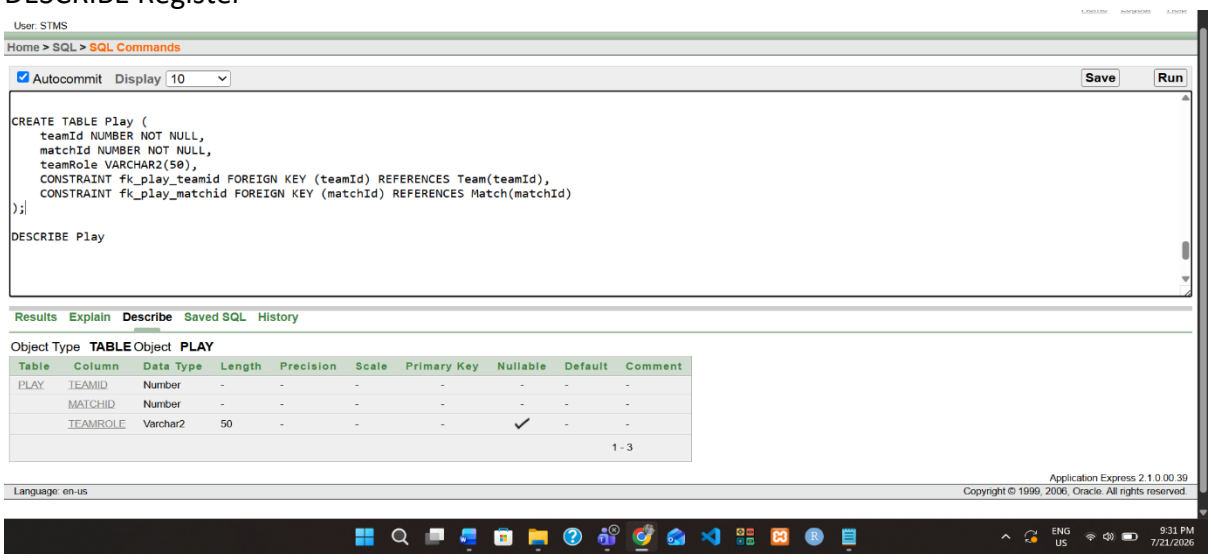
Fig

9. CREATE TABLE Organize (
   userId NUMBER NOT NULL,
   tournamentId NUMBER NOT NULL,

CONSTRAINT fk_organize_userid FOREIGN KEY (userId) REFERENCES USERs (userId),
CONSTRAINT fk_organize_tournid FOREIGN KEY (tournamentId) REFERENCES Tournament(tournamentId)
);
DESCRIBE Organize

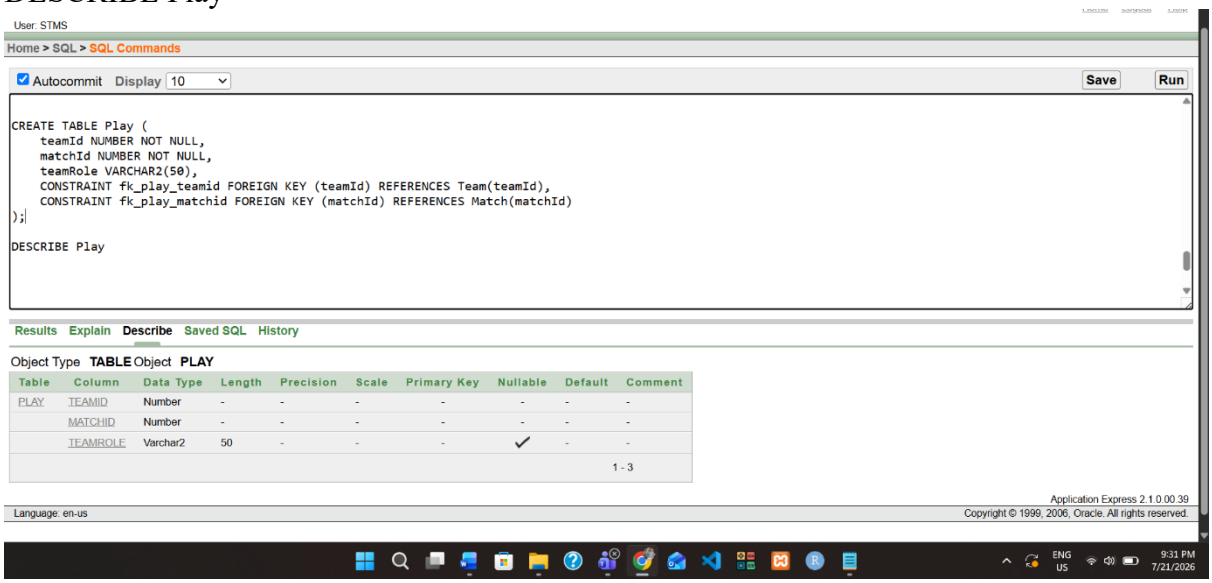


10. CREATE TABLE Register (
teamId NUMBER NOT NULL,
tournamentId NUMBER NOT NULL,
CONSTRAINT fk_register_teamid FOREIGN KEY (teamId) REFERENCES Team(teamId),
CONSTRAINT fk_register_tournid FOREIGN KEY (tournamentId) REFERENCES Tournament(tournamentId)
);
DESCRIBE Register

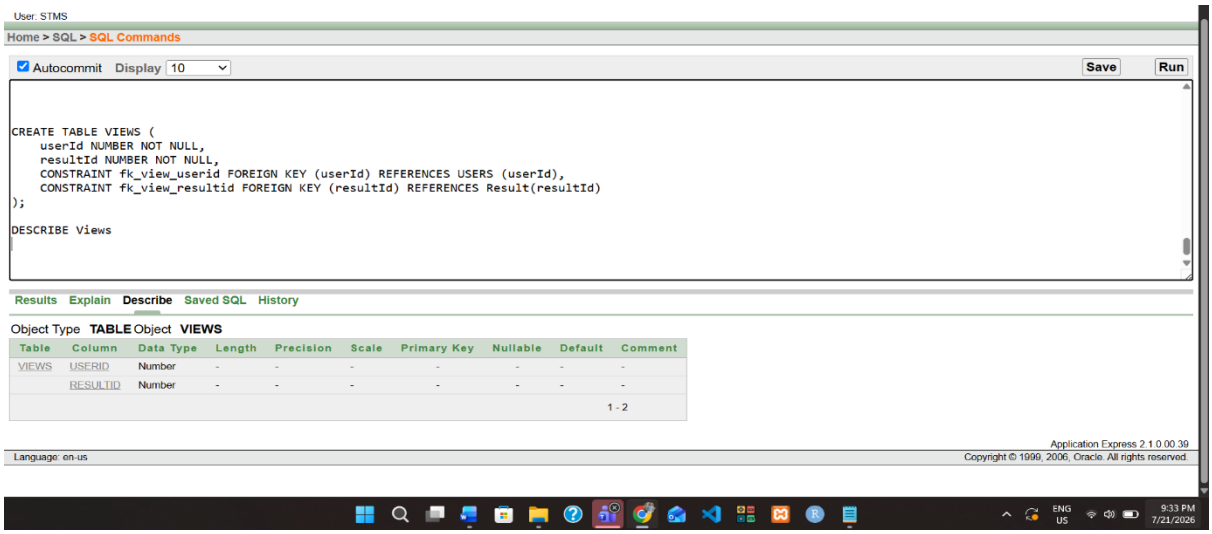


11. CREATE TABLE Play (
teamId NUMBER NOT NULL,
matchId NUMBER NOT NULL,
teamRole VARCHAR2(50),
CONSTRAINT fk_play_teamid FOREIGN KEY (teamId) REFERENCES Team(teamId),

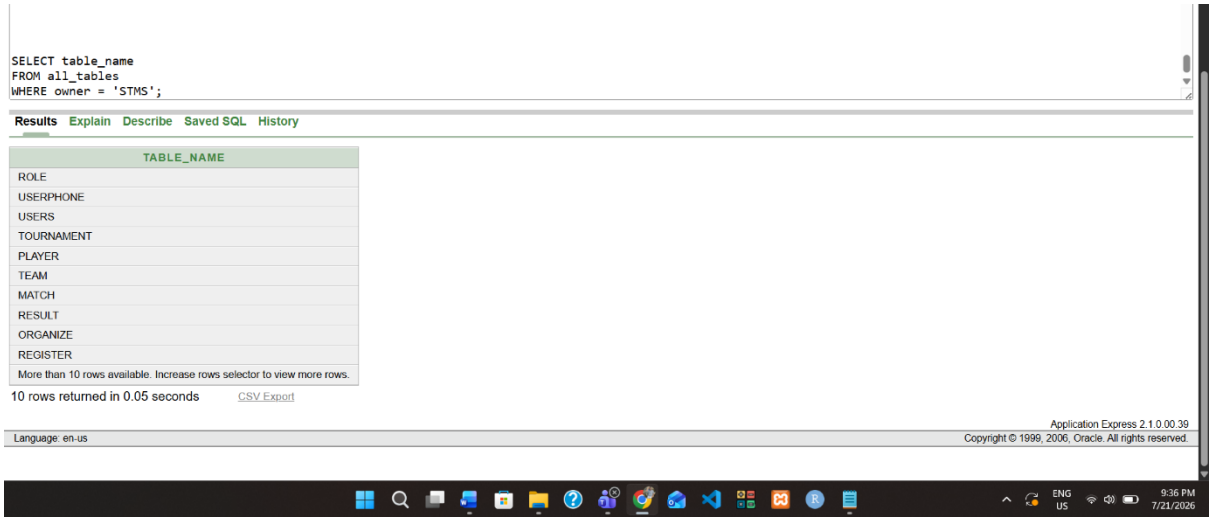
CONSTRAINT fk_play_matchid FOREIGN KEY (matchId) REFERENCES Match(matchId)
);
DESCRIBE Play



12. CREATE TABLE VIEWS (
userId NUMBER NOT NULL,
resultId NUMBER NOT NULL,
CONSTRAINT fk_view_userid FOREIGN KEY (userId) REFERENCES USERS (userId),
CONSTRAINT fk_view_resultid FOREIGN KEY (resultId) REFERENCES Result(resultId)
);
DESCRIBE Views



```
SELECT table_name
FROM all_tables
WHERE owner = 'STMS';
```

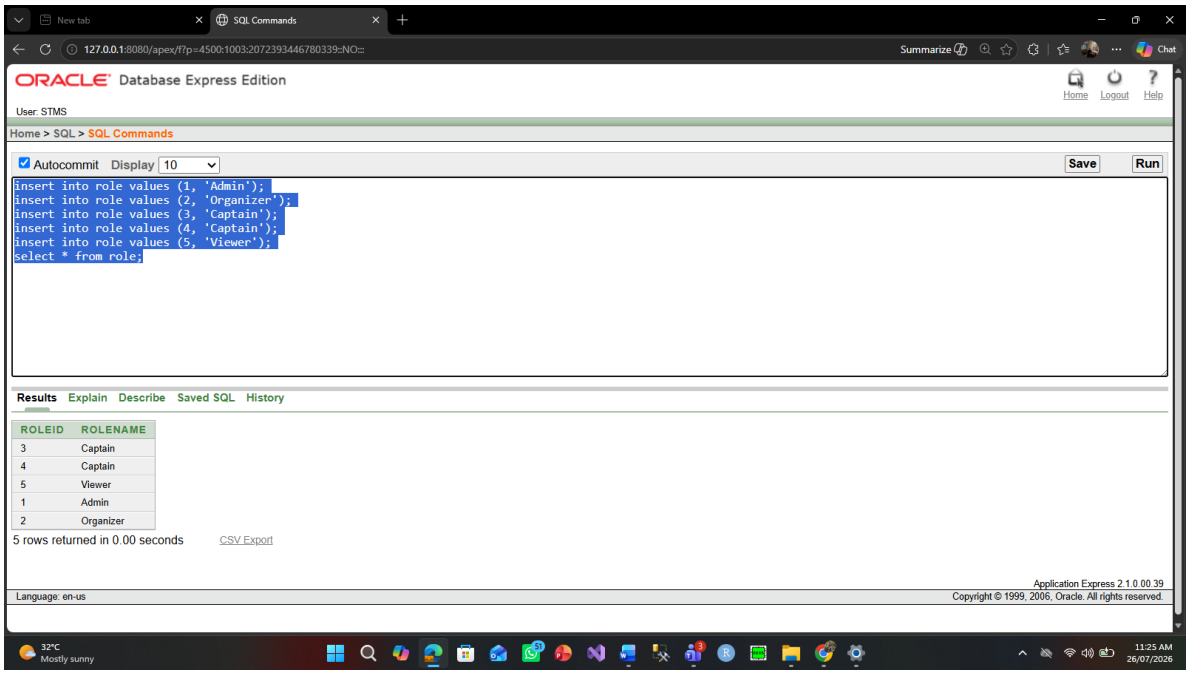


Data Insertion Using SQL

Data Insertion Using SQL

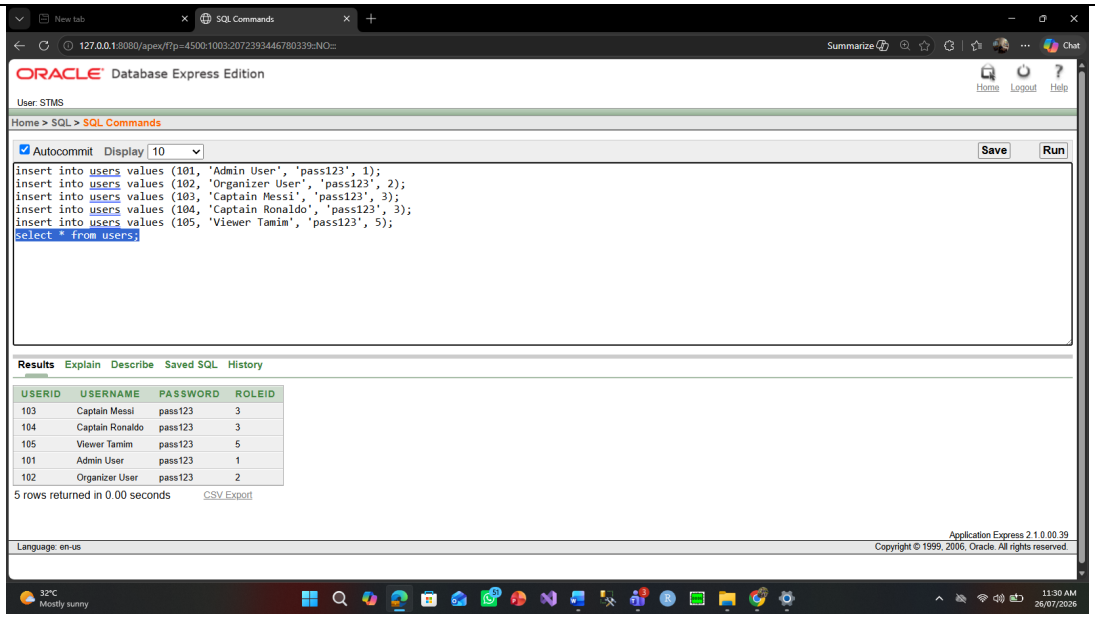
1. Insertion for Role:

```
insert into role values (1, 'Admin');
insert into role values (2, 'Organizer');
insert into role values (3, 'Captain');
insert into role values (4, 'Captain');
insert into role values (5, 'Viewer');
select * from role;
```



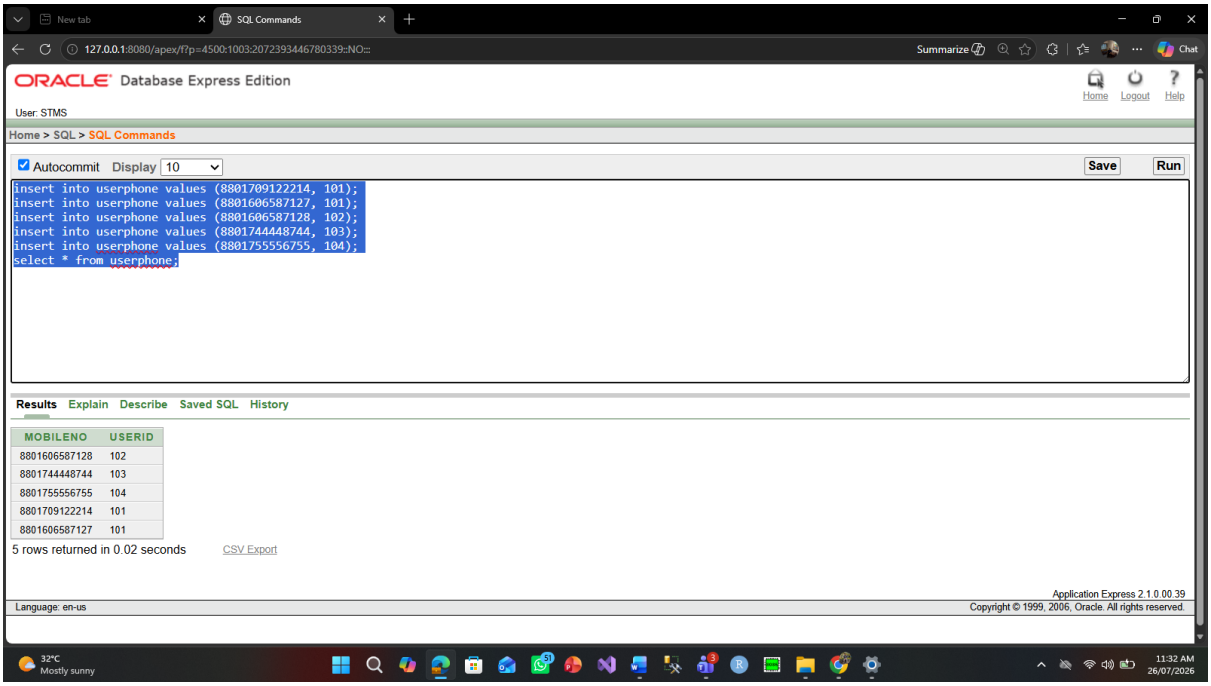
2. Insertion for Users:

```
insert into users values (101, 'Admin User', 'pass123', 1);
insert into users values (102, 'Organizer User', 'pass123', 2);
insert into users values (103, 'Captain Messi', 'pass123', 3);
insert into users values (104, 'Captain Ronaldo', 'pass123', 3);
insert into users values (105, 'Viewer Tamim', 'pass123', 5);
select * from users;
```



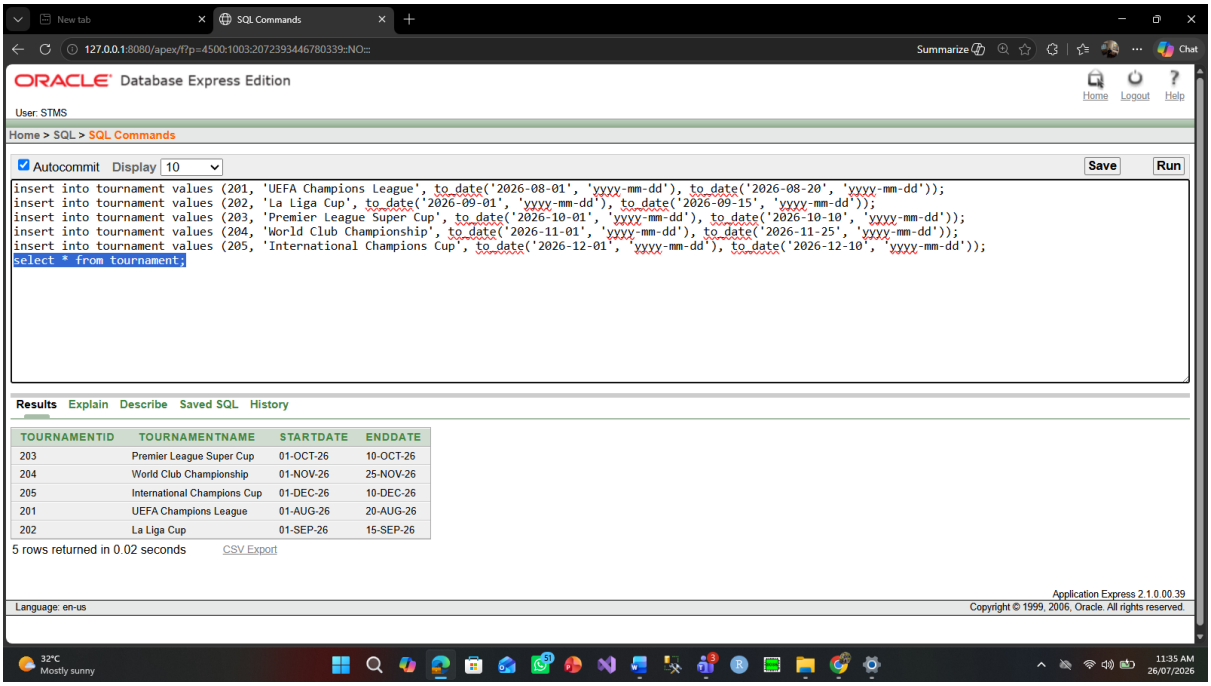
3. Insertion for UserPhone:

```
insert into userphone values (8801709122214, 101);
insert into userphone values (8801606587127, 101);
insert into userphone values (8801606587128, 102);
insert into userphone values (8801744448744, 103);
insert into userphone values (8801755556755, 104);
select * from userphone;
```



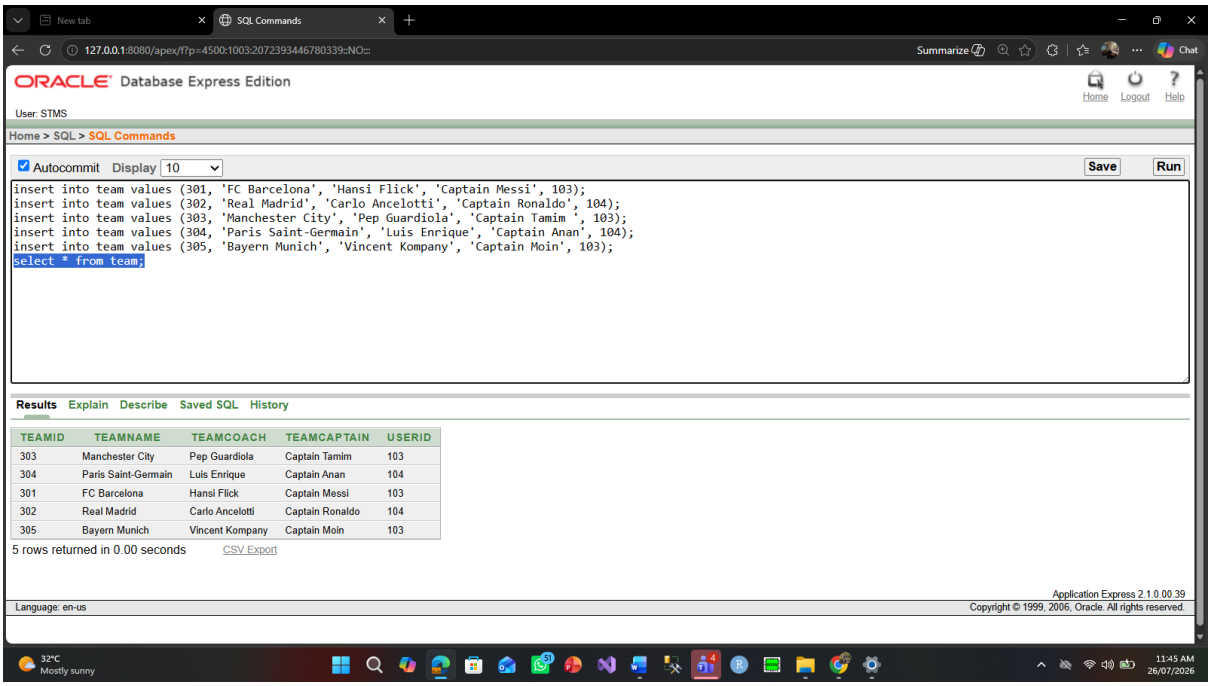
4. Insertion for Tournament:

```
insert into tournament values (201, 'UEFA Champions League', to_date('2026-08-01', 'yyyy-mm-dd'), to_date('2026-08-20', 'yyyy-mm-dd'));
insert into tournament values (202, 'La Liga Cup', to_date('2026-09-01', 'yyyy-mm-dd'), to_date('2026-09-15', 'yyyy-mm-dd'));
insert into tournament values (203, 'Premier League Super Cup', to_date('2026-10-01', 'yyyy-mm-dd'), to_date('2026-10-10', 'yyyy-mm-dd'));
insert into tournament values (204, 'World Club Championship', to_date('2026-11-01', 'yyyy-mm-dd'), to_date('2026-11-25', 'yyyy-mm-dd'));
insert into tournament values (205, 'International Champions Cup', to_date('2026-12-01', 'yyyy-mm-dd'), to_date('2026-12-10', 'yyyy-mm-dd'));
select * from tournament;
```



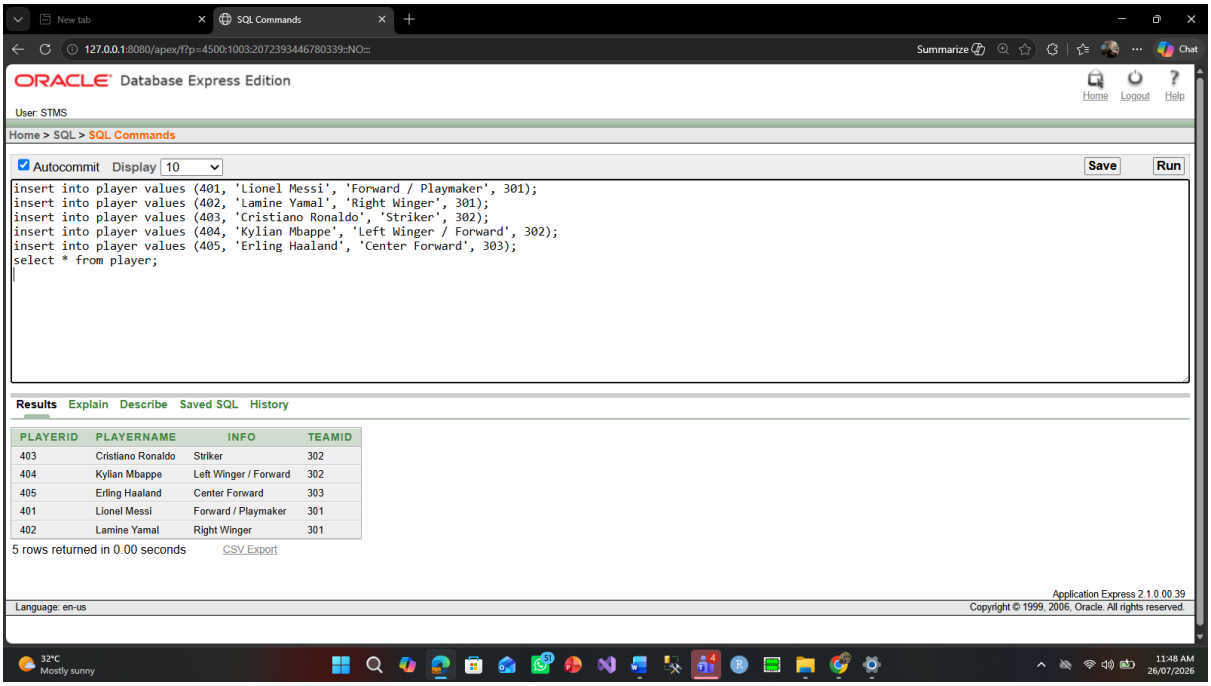
5. Insertion for Team:

```
insert into team values (301, 'FC Barcelona', 'Hansi Flick', 'Captain Messi', 103);
insert into team values (302, 'Real Madrid', 'Carlo Ancelotti', 'Captain Ronaldo', 104);
insert into team values (303, 'Manchester City', 'Pep Guardiola', 'Captain Tamim ', 103);
insert into team values (304, 'Paris Saint-Germain', 'Luis Enrique', 'Captain Anan', 104);
insert into team values (305, 'Bayern Munich', 'Vincent Kompany', 'Captain Moin', 103);
select * from team;
```



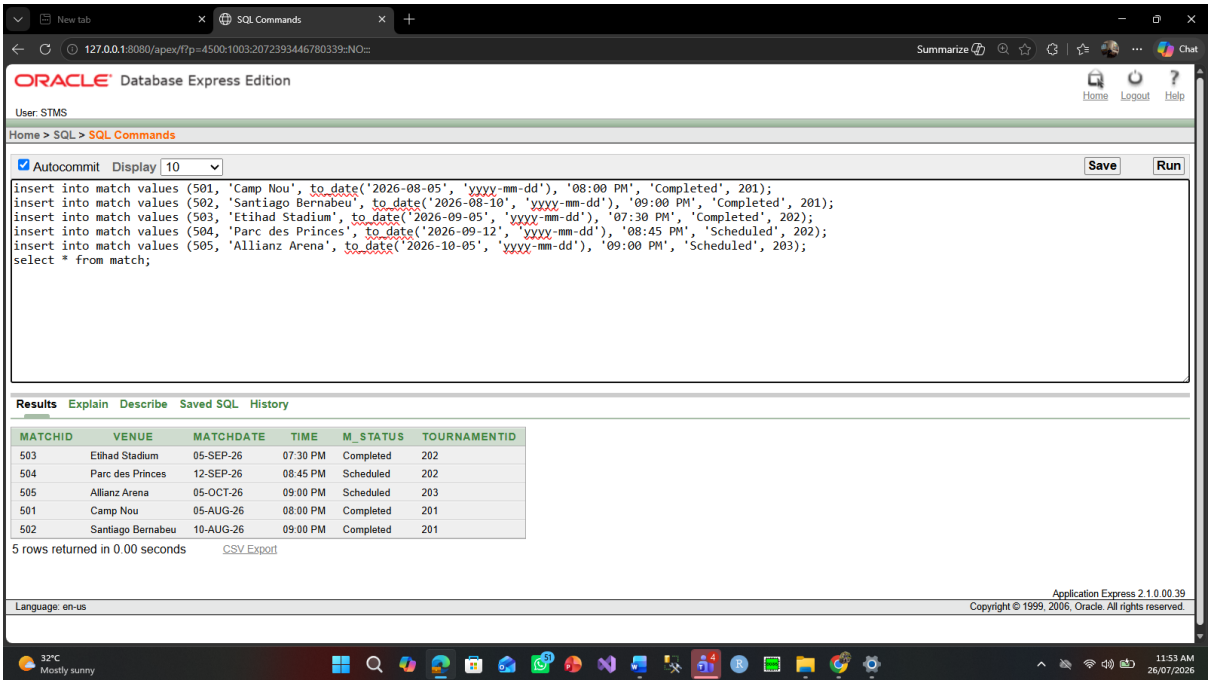
6. Insertion for Player:

```
insert into player values (401, 'Lionel Messi', 'Forward / Playmaker', 301);
insert into player values (402, 'Lamine Yamal', 'Right Winger', 301);
insert into player values (403, 'Cristiano Ronaldo', 'Striker', 302);
insert into player values (404, 'Kylian Mbappe', 'Left Winger / Forward', 302);
insert into player values (405, 'Erling Haaland', 'Center Forward', 303);
select * from player;
```



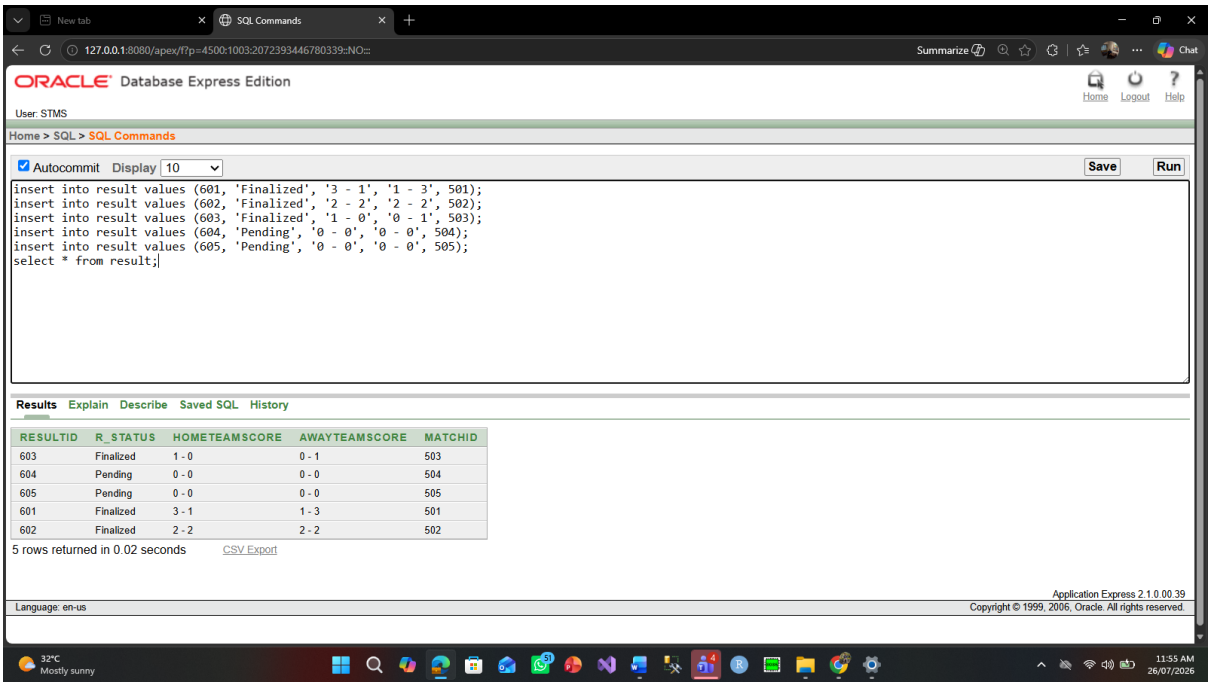
7. Insertion for Match:

```
insert into match values (501, 'Camp Nou', to_date('2026-08-05', 'yyyy-mm-dd'), '08:00 PM', 'Completed', 201);
insert into match values (502, 'Santiago Bernabeu', to_date('2026-08-10', 'yyyy-mm-dd'), '09:00 PM', 'Completed', 201);
insert into match values (503, 'Etihad Stadium', to_date('2026-09-05', 'yyyy-mm-dd'), '07:30 PM', 'Completed', 202);
insert into match values (504, 'Parc des Princes', to_date('2026-09-12', 'yyyy-mm-dd'), '08:45 PM', 'Scheduled', 202);
insert into match values (505, 'Allianz Arena', to_date('2026-10-05', 'yyyy-mm-dd'), '09:00 PM', 'Scheduled', 203);
select * from match;
```



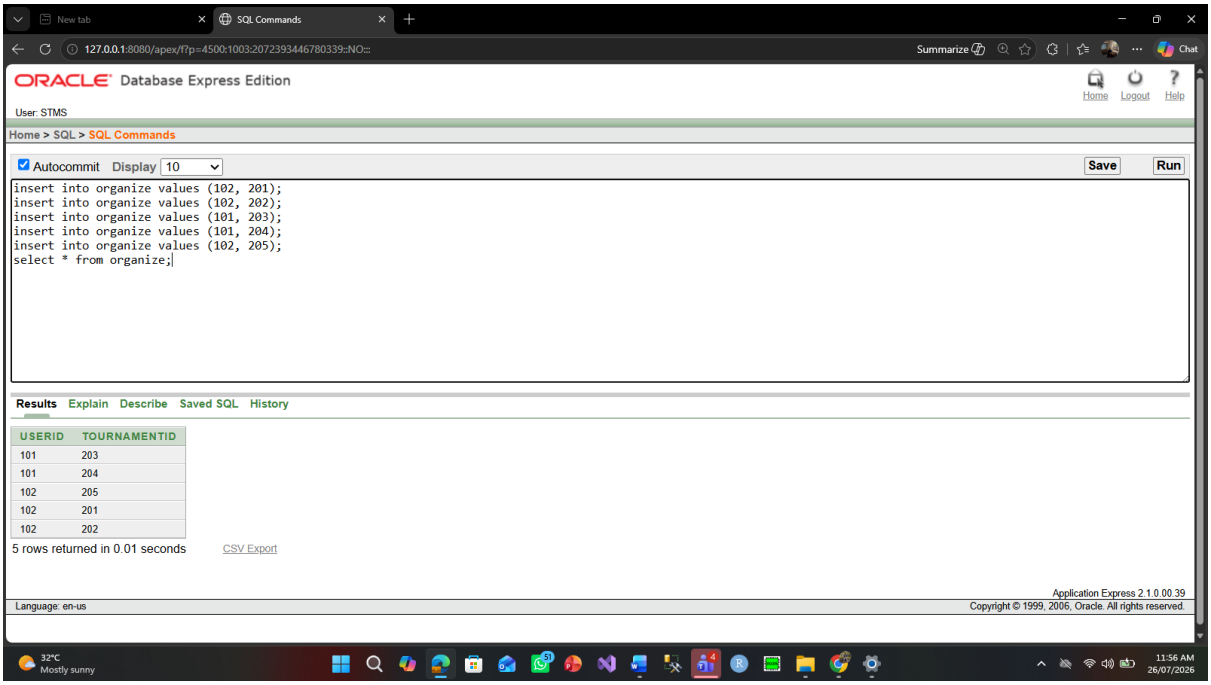
8. Insertion for Result:

```
insert into result values (601, 'Finalized', '3 - 1', '1 - 3', 501);
insert into result values (602, 'Finalized', '2 - 2', '2 - 2', 502);
insert into result values (603, 'Finalized', '1 - 0', '0 - 1', 503);
insert into result values (604, 'Pending', '0 - 0', '0 - 0', 504);
insert into result values (605, 'Pending', '0 - 0', '0 - 0', 505);
select * from result
```



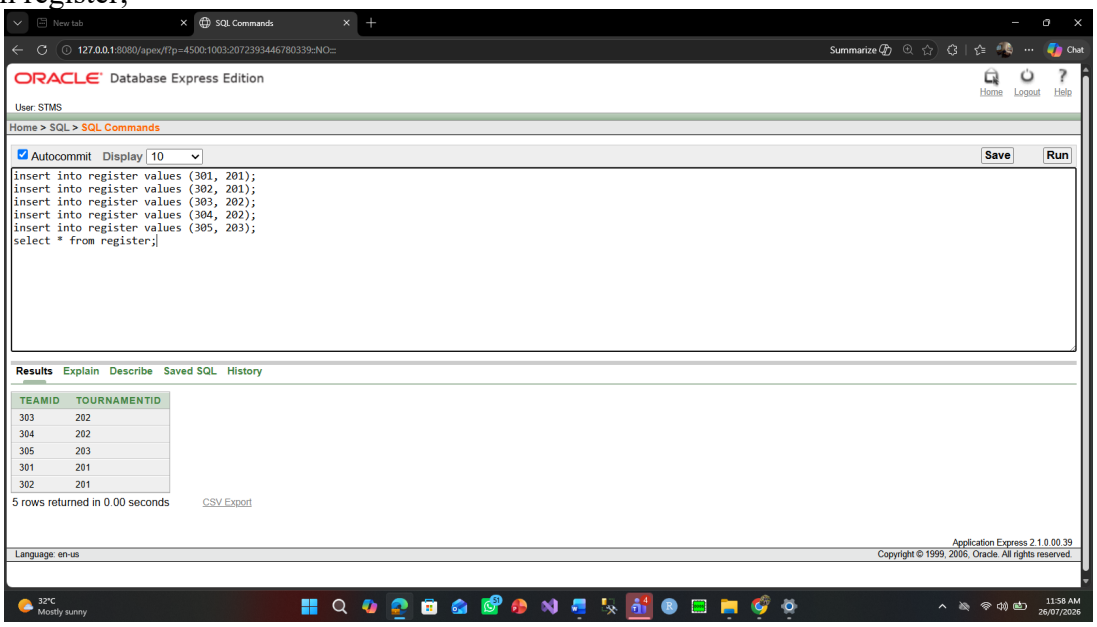
9. Insertion for Organize:

```
insert into organize values (102, 201);
insert into organize values (102, 202);
insert into organize values (101, 203);
insert into organize values (101, 204);
insert into organize values (102, 205);
select * from organize
```



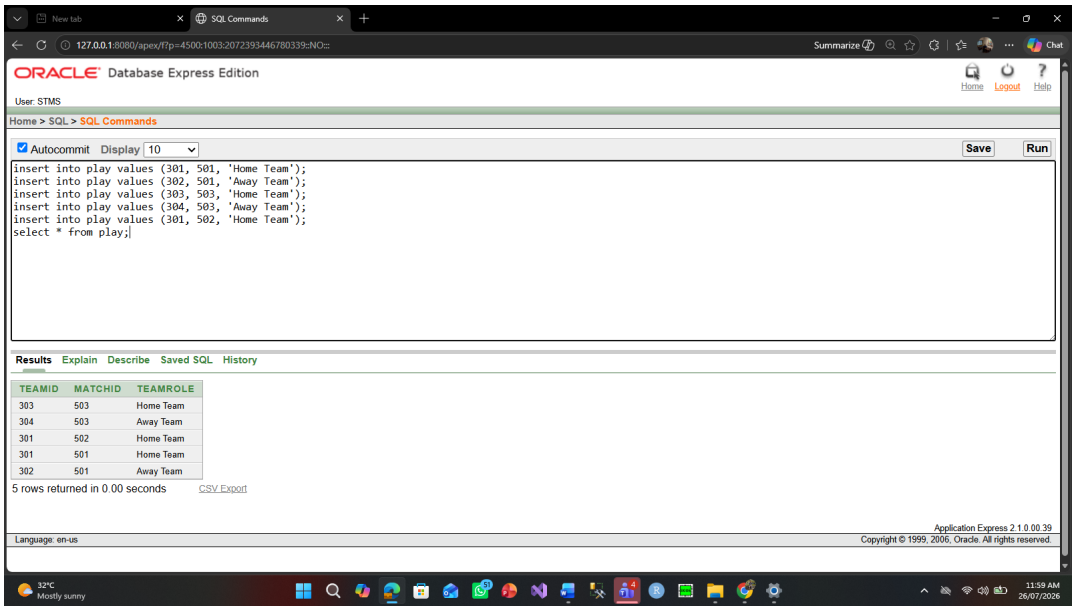
10. Insertion for Register:

```
insert into register values (301, 201);
insert into register values (302, 201);
insert into register values (303, 202);
insert into register values (304, 202);
insert into register values (305, 203);
select * from register;
```



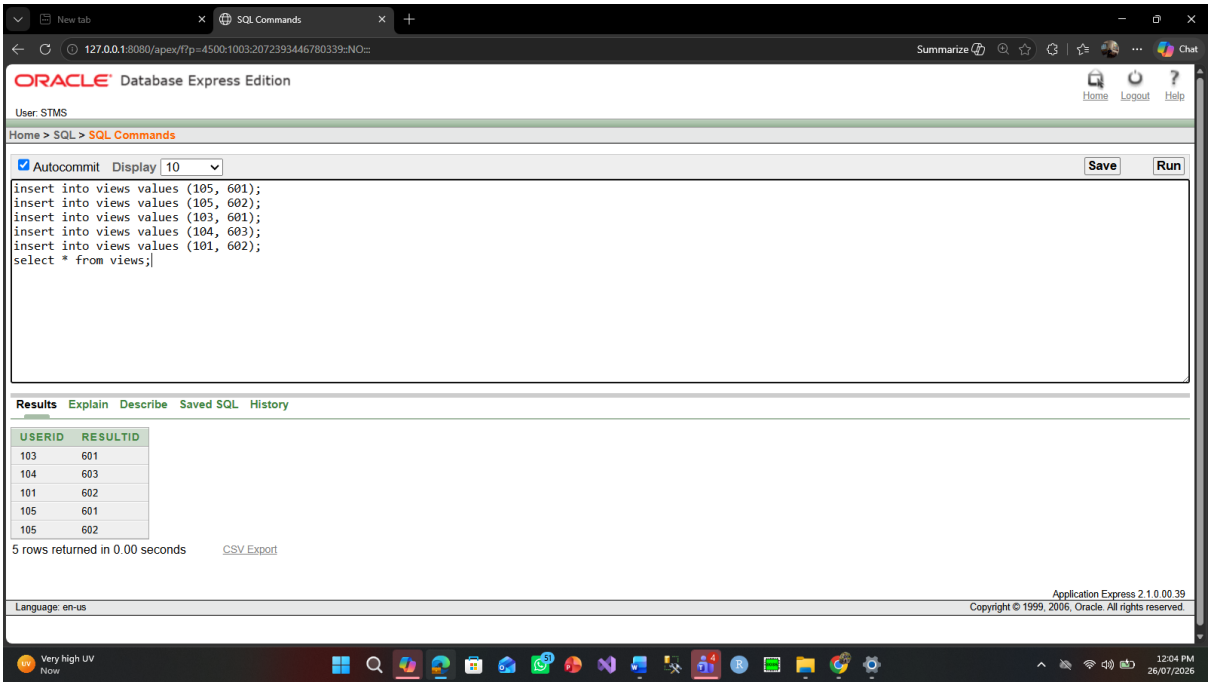
11. Insertion for Play:

```
insert into play values (301, 501, 'Home Team');
insert into play values (302, 501, 'Away Team');
insert into play values (303, 503, 'Home Team');
insert into play values (304, 503, 'Away Team');
insert into play values (301, 502, 'Home Team');
select * from play;
```



12. Insertion for Views:

```
insert into views values (105, 601);
insert into views values (105, 602);
insert into views values (103, 601);
insert into views values (104, 603);
insert into views values (101, 602);
select * from views;
```



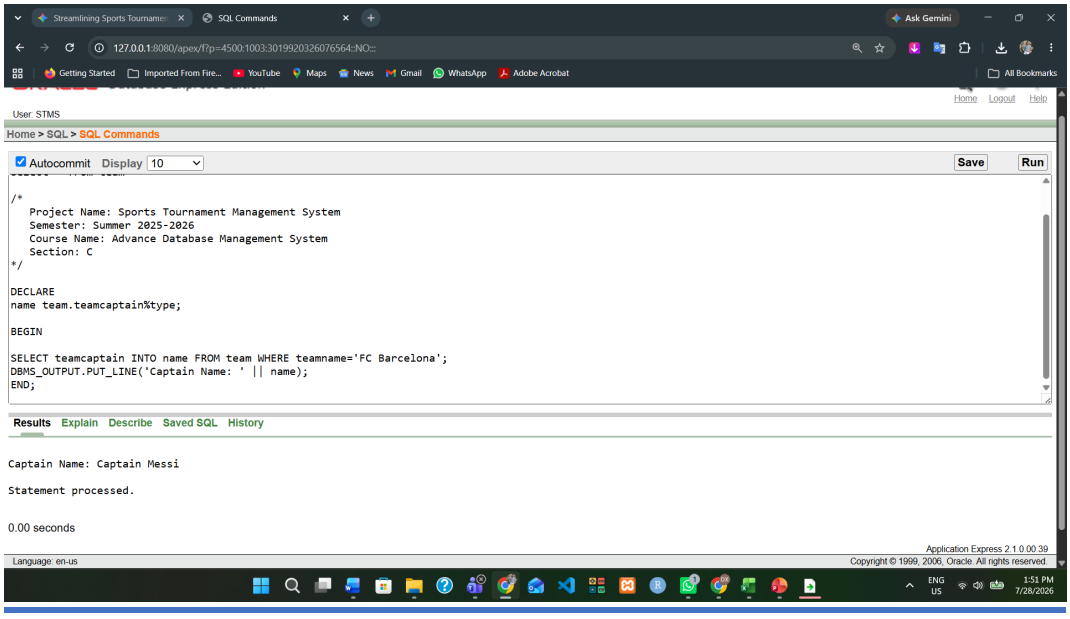
Query Basic PL/SQL

Variable

Question 1:

Write a pl/sql code that shows the FC Barcelona's captain name.

```
/*
Project Name: Sports Tournament Management System

  Semester: Summer 2025-2026

  Course Name: Advance Database Management System

  Section: C

*/

DECLARE
name team.teamcaptain%type;

BEGIN

SELECT teamcaptain INTO name FROM team WHERE teamname='FC
Barcelona';DBMS_OUTPUT.PUT_LINE('Captain Name: ' || name);

END;
```

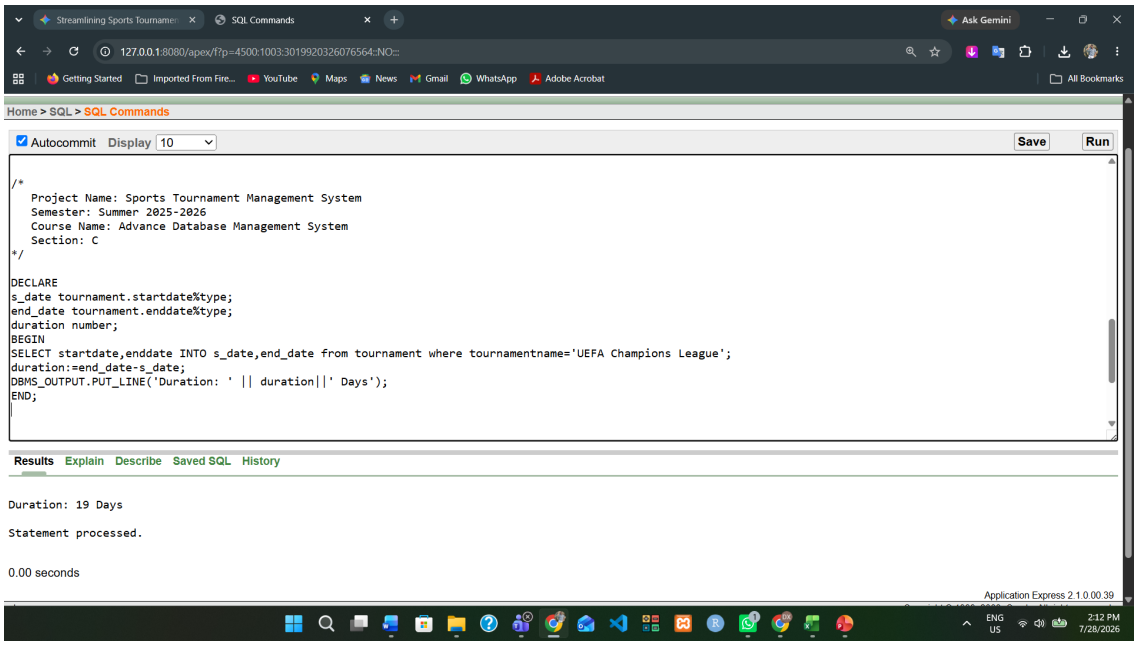


Operators

Question 3:

Show duration of the tournament named UEFA Champions League by using pl/sql

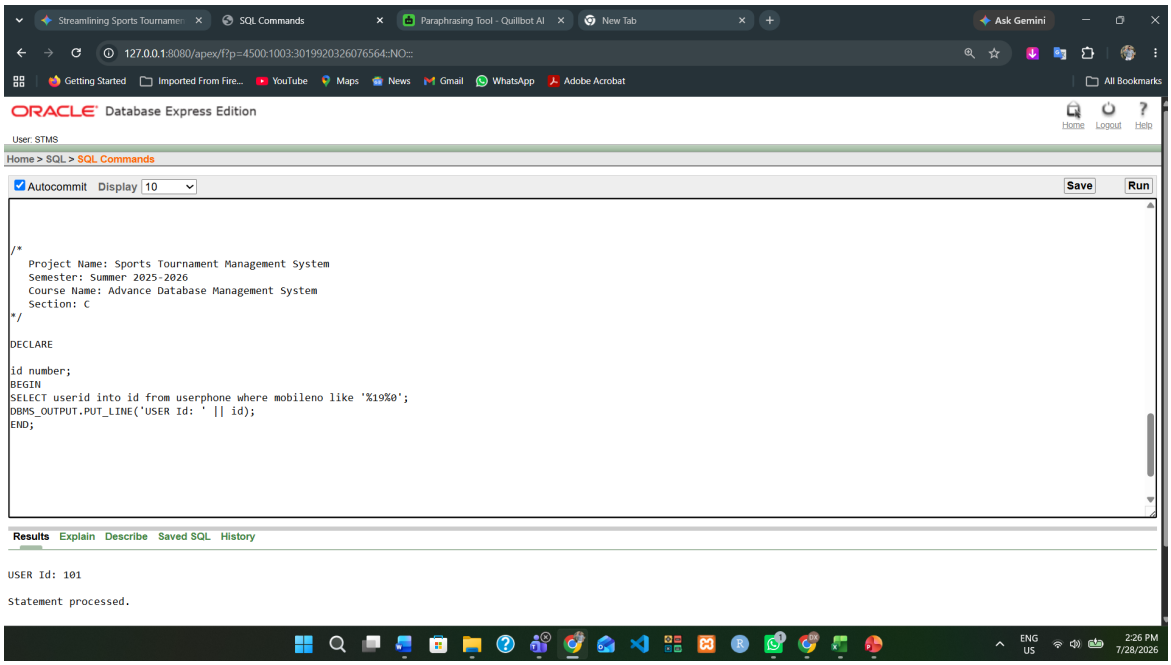
```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/

DECLARE
s_date tournament.startdate%type;
end_date tournament.enddate%type;
duration number;
BEGIN
SELECT startdate,enddate INTO s_date,end_date from tournament where tournamentname='UEFA
 Champions League';
duration:=end_date-s_date;
DBMS_OUTPUT.PUT_LINE('Duration: ' || duration||' Days');
 END;
```



Question 4:

Create a pl/sql code that displays the userid whose mobile number is like '%19%0'.

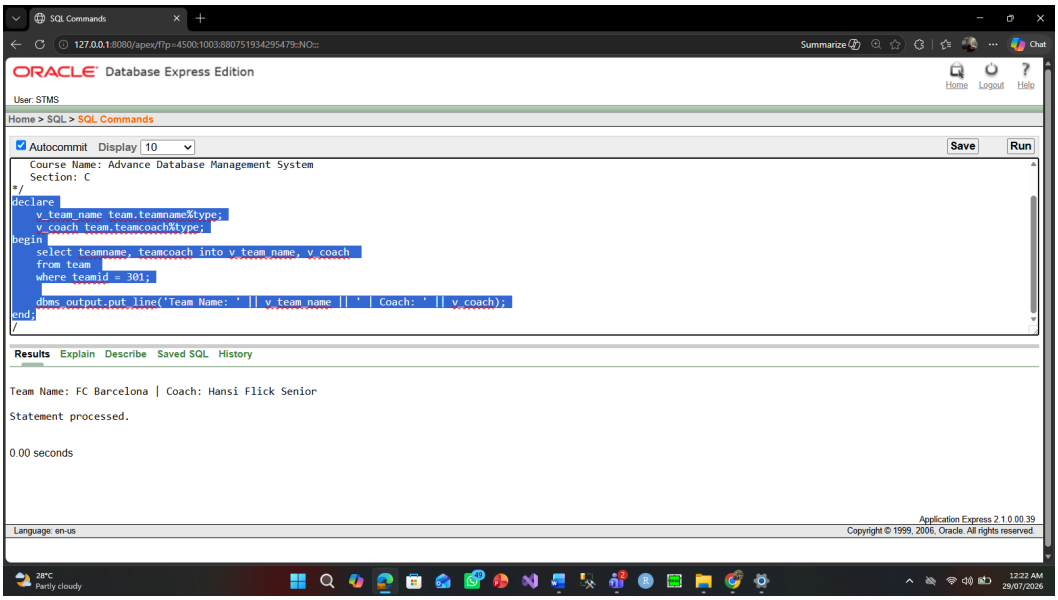
```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/

DECLARE

id number;
BEGIN
SELECT userid into id from userphone where mobileno like '%19%0';
DBMS_OUTPUT.PUT_LINE('USER Id: ' || id);
END;
```



Question 5:

Write a PL/SQL code that displays the name and coach of the team with ID 301.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_team_name team.teamname%type;
   v_coach team.teamcoach%type;
begin
   select teamname, teamcoach into v_team_name, v_coach
   from team
   where teamid = 301;

   dbms_output.put_line('Team Name: ' || v_team_name || ' | Coach: ' || v_coach);
end;
/
```

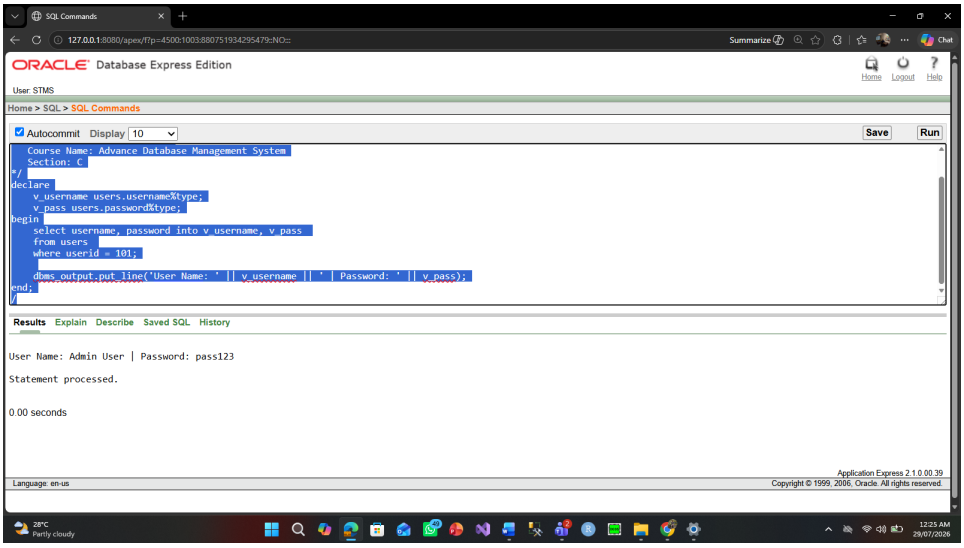


Question 6:

Write a PL/SQL code to display the user name and password for user ID 101.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_username users.username%type;
```

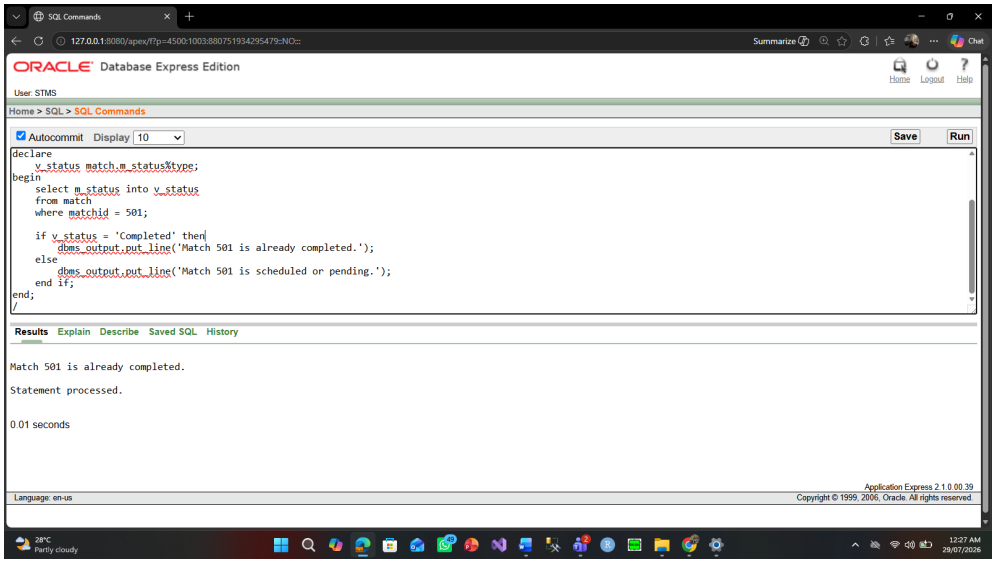
```
    v_pass users.password%type;
begin
    select username, password into v_username, v_pass
    from users
    where userid = 101;

    dbms_output.put_line('User Name: ' || v_username || ' | Password: ' || v_pass);
end;
                                                    /
```



Question 7:
Write a PL/SQL code to check whether the match status for match ID 501 is 'Completed' or 'Scheduled' using IF-THEN-ELSE.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
    v_status match.m_status%type;
begin
    select m_status into v_status
    from match
    where matchid = 501;

    if v_status = 'Completed' then
        dbms_output.put_line('Match 501 is already completed.');
    else
        dbms_output.put_line('Match 501 is scheduled or pending.');
    end if;
end;
```

/

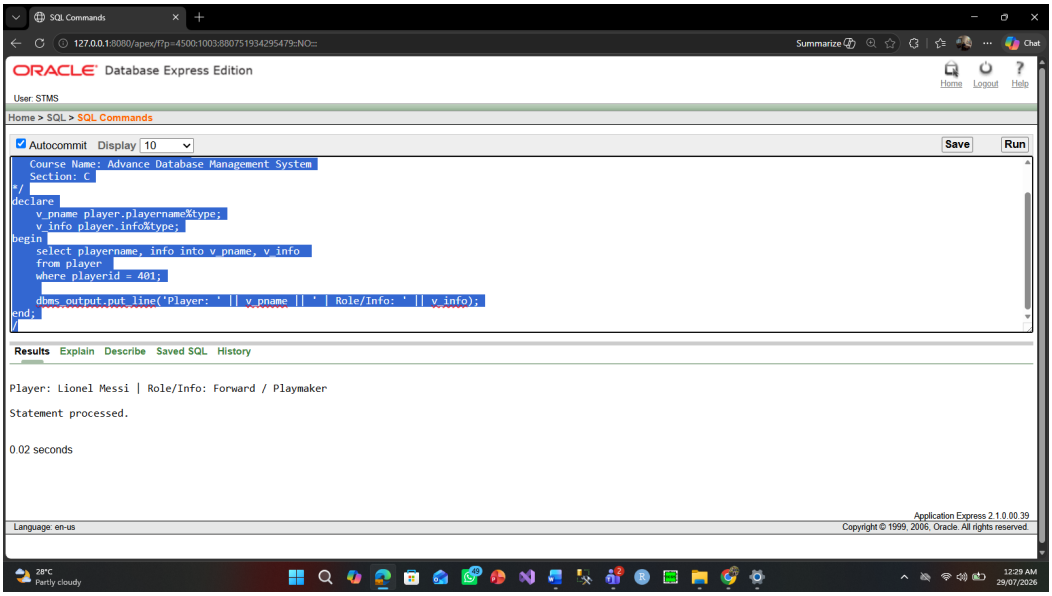


Question 8:

Write a PL/SQL code to fetch the player name and position info for player ID 401.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_pname player.playername%type;
   v_info player.info%type;
begin
   select playername, info into v_pname, v_info
   from player
   where playerid = 401;

   dbms_output.put_line('Player: ' || v_pname || ' | Role/Info: ' || v_info);
end;
```

/

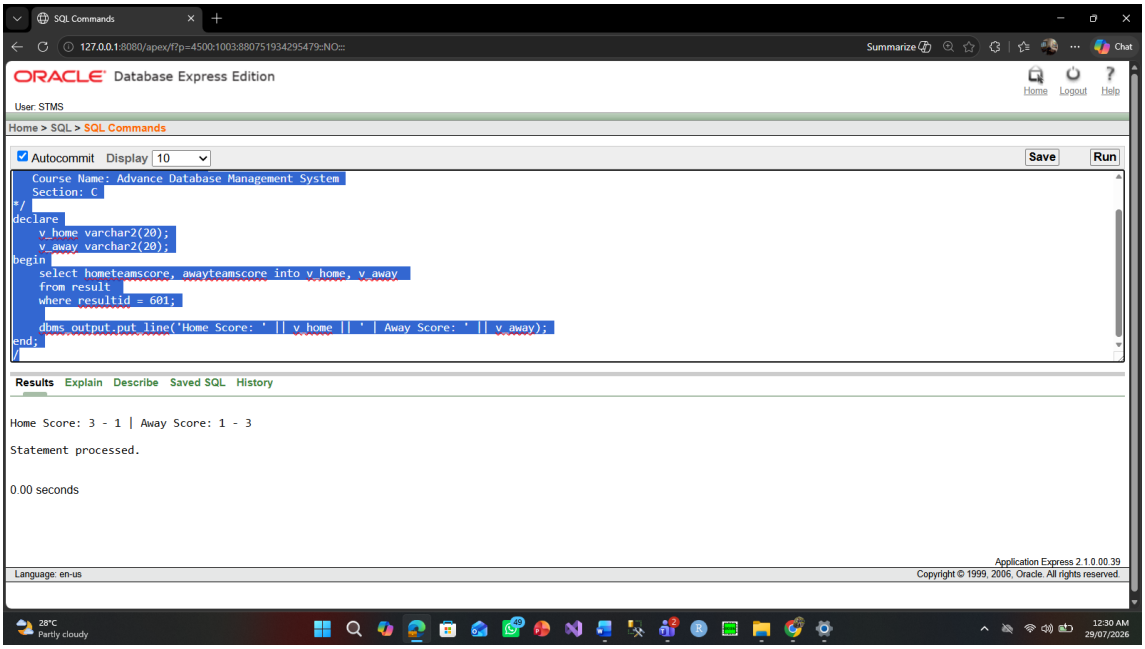


Question 9:

Write a PL/SQL code to calculate total home team score and away team score sum for result ID 601.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_home varchar2(20);
   v_away varchar2(20);
begin
   select hometeamscore, awayteamscore into v_home, v_away
   from result
   where resultid = 601;

   dbms_output.put_line('Home Score: ' || v_home || ' | Away Score: ' || v_away);
end;
```

/

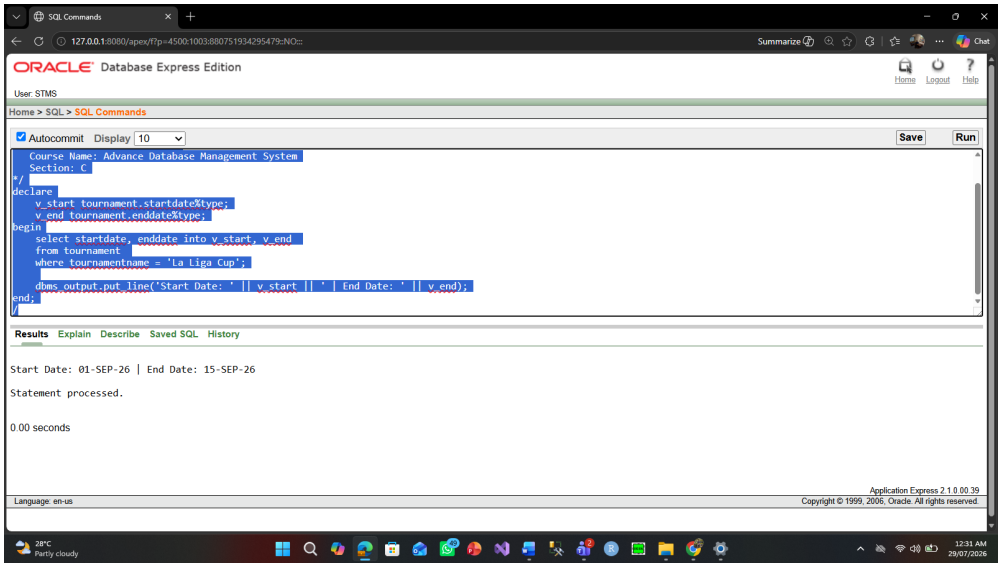


Question 10:

Write a PL/SQL code to find the start date and end date of the tournament named 'La Liga Cup'.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_start tournament.startdate%type;
   v_end tournament.enddate%type;
begin
   select startdate, enddate into v_start, v_end
   from tournament
   where tournamentname = 'La Liga Cup';

   dbms_output.put_line('Start Date: ' || v_start || ' | End Date: ' || v_end);
end;
```

/

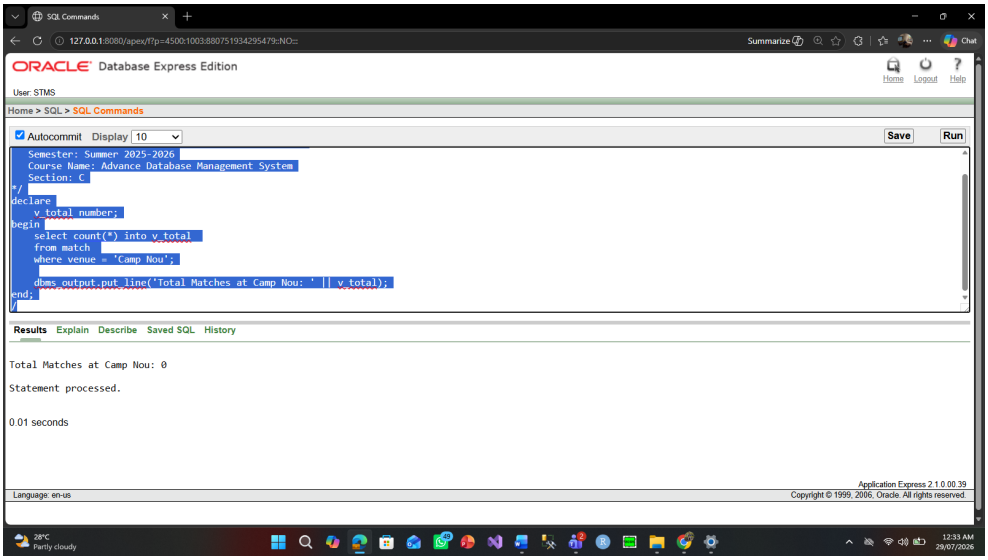


Question 11:

Write a PL/SQL code to count total matches scheduled in Camp Nou venue.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_total number;
begin
   select count(*) into v_total
   from match
   where venue = 'Camp Nou';

   dbms_output.put_line('Total Matches at Camp Nou: ' || v_total);
end;
```

/

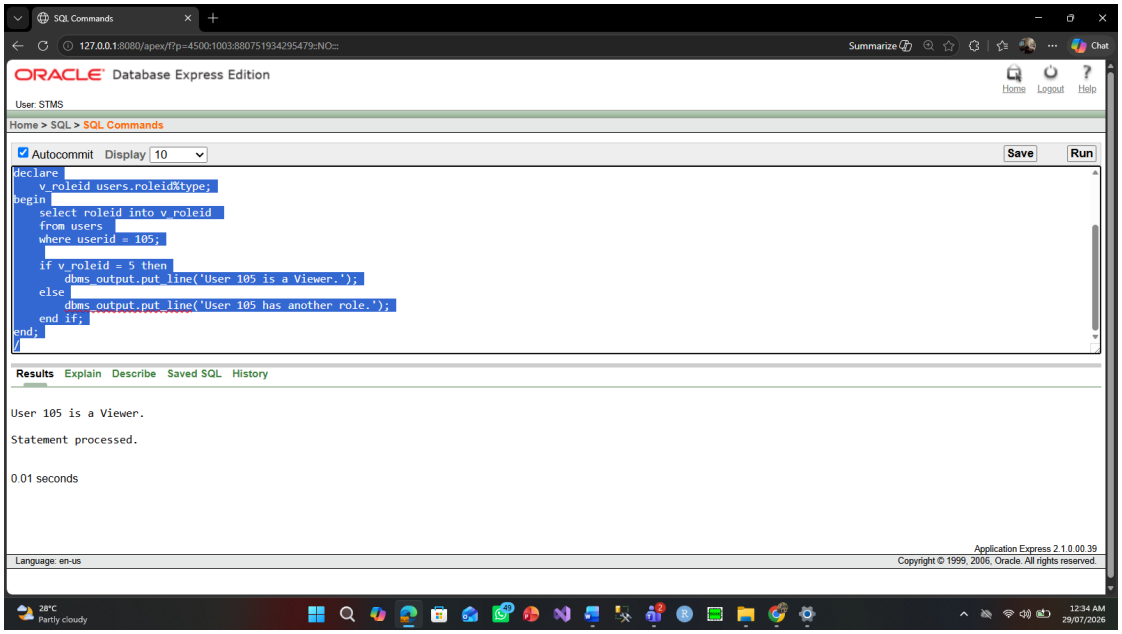


Question 12:

Write a PL/SQL code to check if a user with user ID 105 has 'Viewer' role using IF condition.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_roleid users.roleid%type;
begin
   select roleid into v_roleid
   from users
   where userid = 105;

   if v_roleid = 5 then
      dbms_output.put_line('User 105 is a Viewer.');
   else
      dbms_output.put_line('User 105 has another role.');
   end if;
end;
```

/

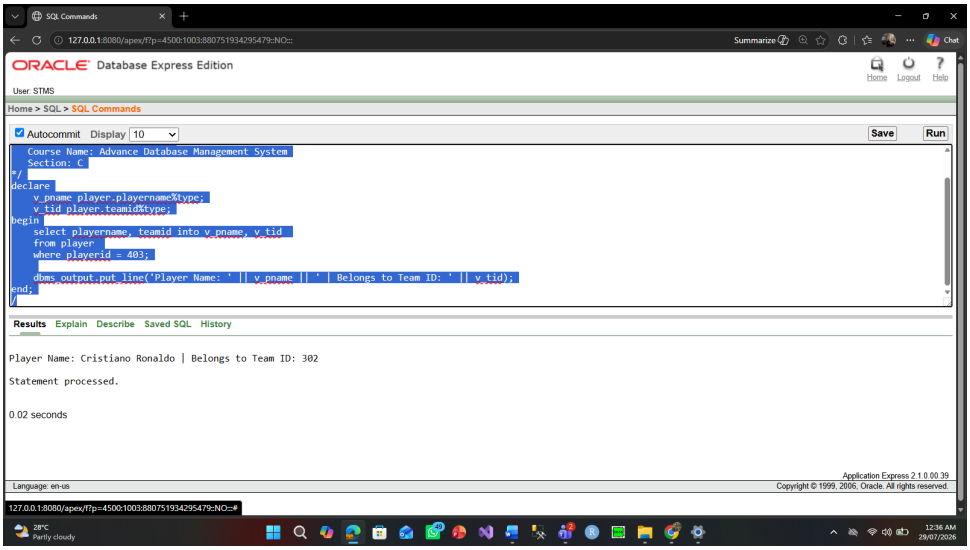


Question 13:

Write a PL/SQL code to display player name and team ID for player ID 403.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_pname player.playername%type;
   v_tid player.teamid%type;
begin
   select playername, teamid into v_pname, v_tid
   from player
   where playerid = 403;

   dbms_output.put_line('Player Name: ' || v_pname || ' | Belongs to Team ID: ' || v_tid);
end;
```

/

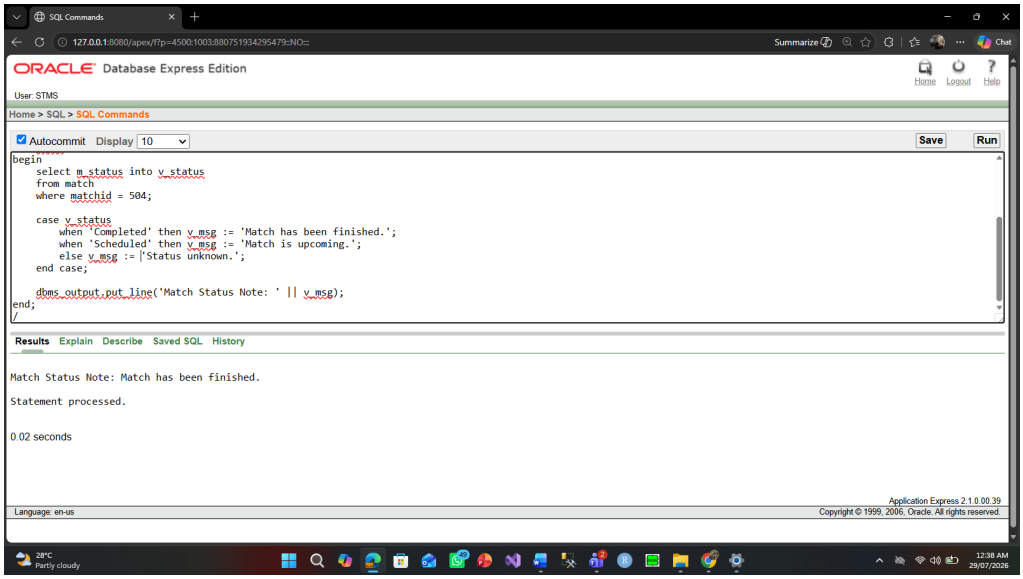


Question 14:

Write a PL/SQL code using CASE statement to describe the match status for match ID 504.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_status match.m_status%type;
   v_msg varchar2(100);
begin
   select m_status into v_status
   from match
   where matchid = 504;

   case v_status
      when 'Completed' then v_msg := 'Match has been finished.';
      when 'Scheduled' then v_msg := 'Match is upcoming.';
      else v_msg := 'Status unknown.';
   end case;

   dbms_output.put_line('Match Status Note: ' || v_msg);
end;
```

/

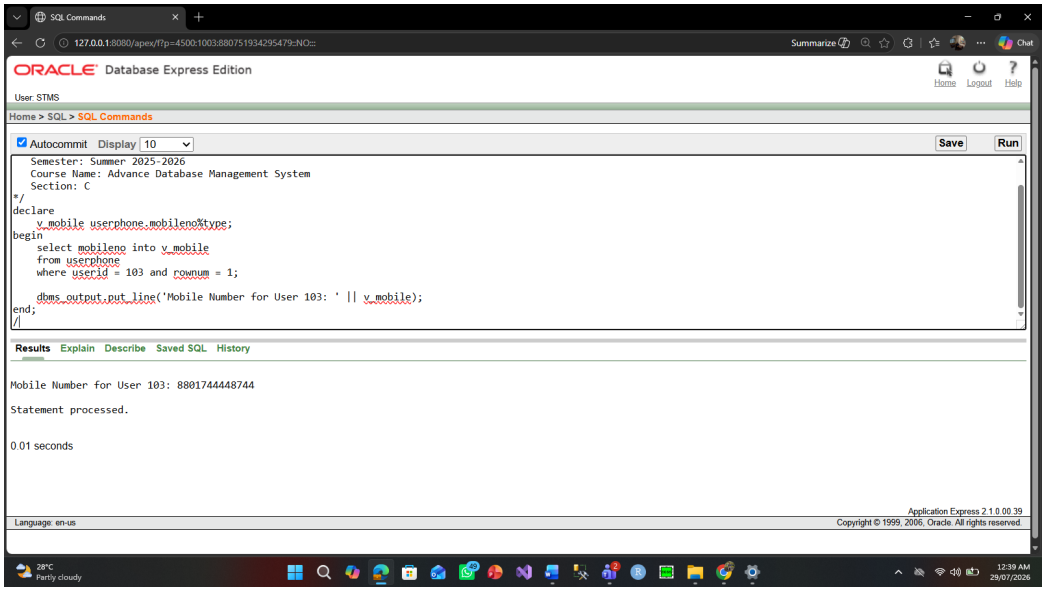


Question 15:

Write a PL/SQL code to fetch mobile number for user ID 103.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_mobile userphone.mobileno%type;
begin
   select mobileno into v_mobile
   from userphone
   where userid = 103 and rownum = 1;

   dbms_output.put_line('Mobile Number for User 103: ' || v_mobile);
end;
```

/

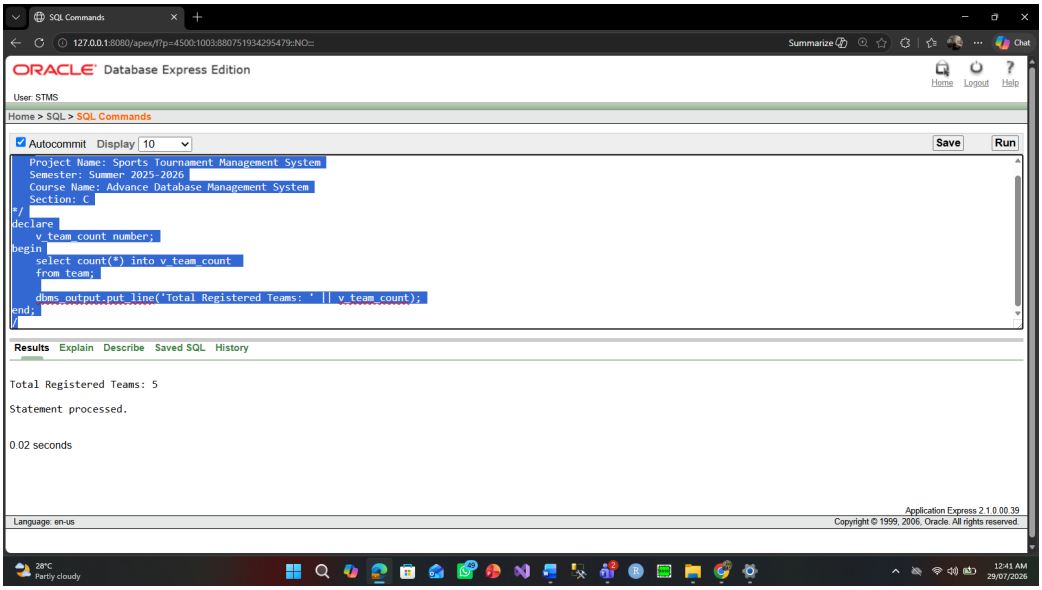


Question 16:

Write a PL/SQL code to calculate total number of teams registered in the system.

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_team_count number;
begin
   select count(*) into v_team_count
   from team;

   dbms_output.put_line('Total Registered Teams: ' || v_team_count);
end;
```

/



Advance PL/SQL

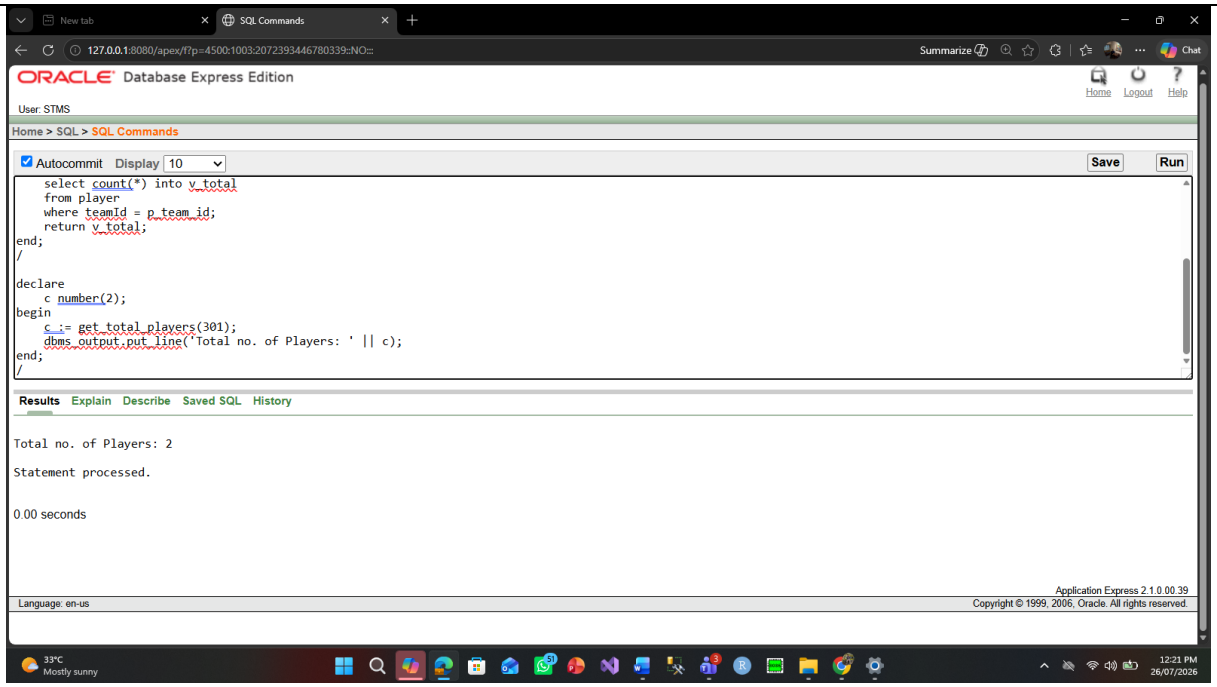
Stored Functions

Question 1:

Write a PL/SQL Stored Function to return the total number of players registered in a specific team.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace function get_total_players(p_team_id number)
return number is
   v_total number := 0;
begin
   select count(*) into v_total
   from player
   where teamId = p_team_id;
   return v_total;
end;
/

declare
   c number(2);
begin
   c := get_total_players(301);
   dbms_output.put_line('Total no. of Players: ' || c);
end;
/
```



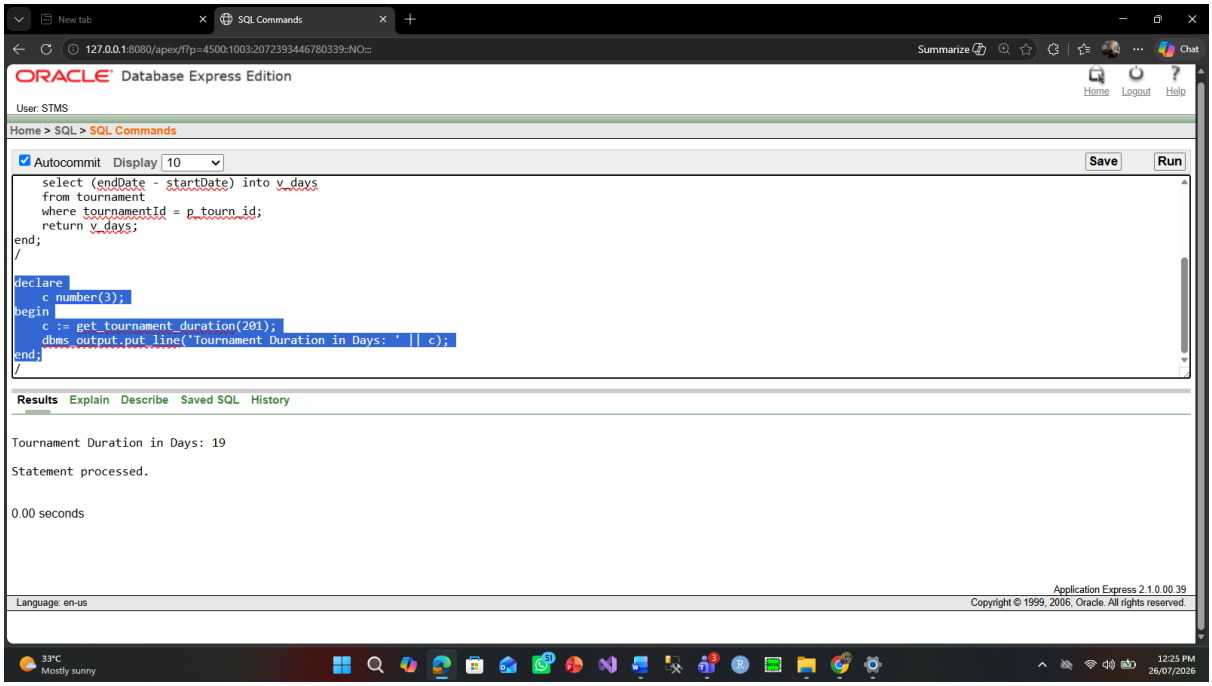
Question 2:

Write a PL/SQL Stored Function to calculate the total duration (in days) of a specific tournament.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace function get_tournament_duration(p_tourn_id number)
return number is
   v_days number := 0;
begin
   select (endDate - startDate) into v_days
   from tournament
   where tournamentId = p_tourn_id;
   return v_days;
end;
/

declare
   c number(3);
begin
   c := get_tournament_duration(201);
   dbms_output.put_line('Tournament Duration in Days: ' || c);
end;
```

/

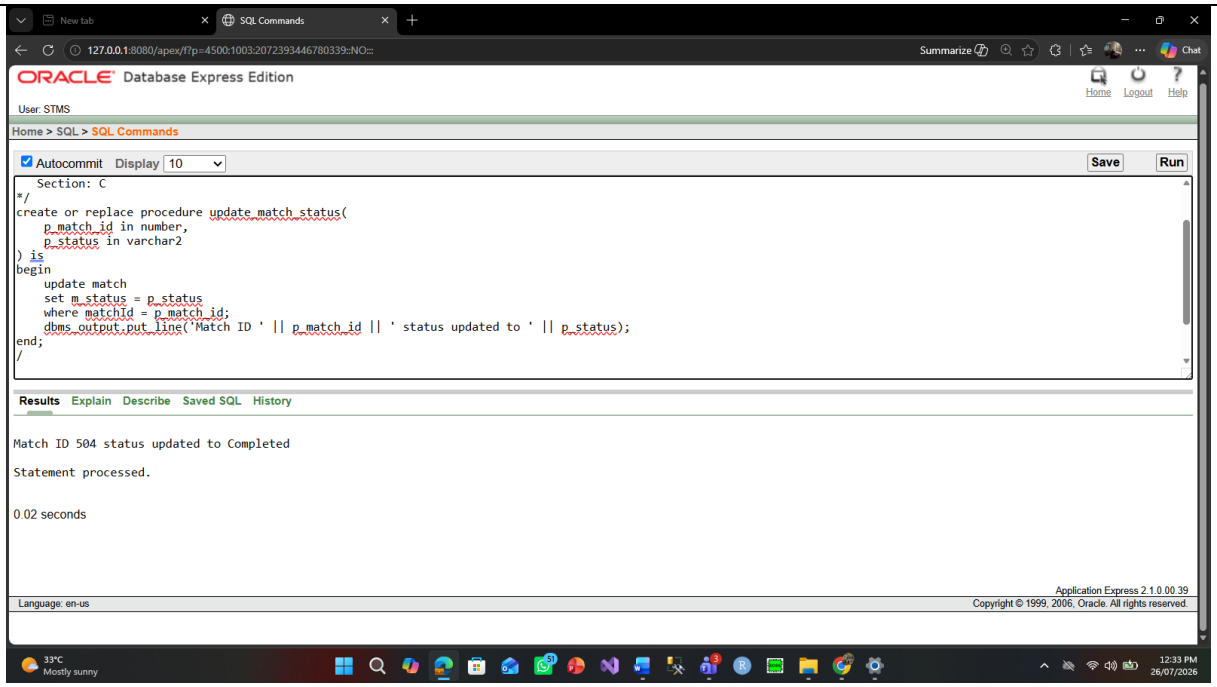


Question 3:

Write a PL/SQL Stored Procedure to update the status of a match based on the provided match.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace procedure update_match_status(
   p_match_id in number,
   p_status in varchar2
) is
begin
   update match
   set m_status = p_status
   where matchId = p_match_id;
   dbms_output.put_line('Match ID ' || p_match_id || ' status updated to ' || p_status);
end;
/

begin
   update_match_status(504, 'Completed');
end;
/
```

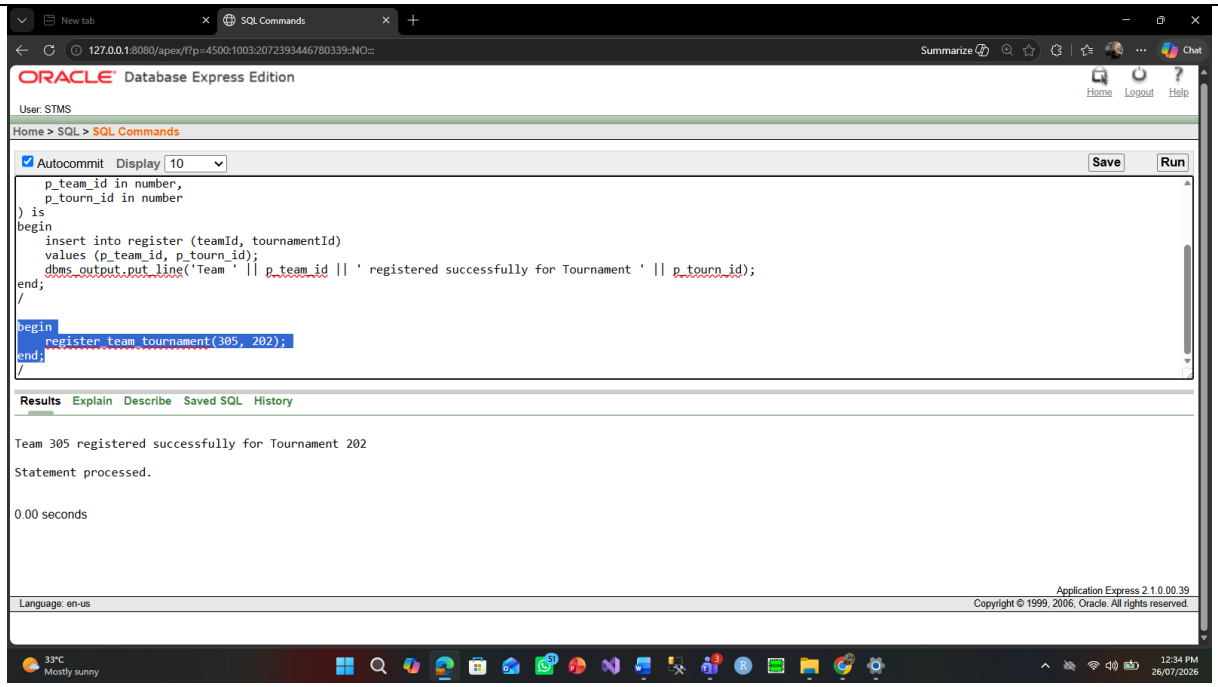


Question 4:

Write a PL/SQL Stored Procedure to register a team for a tournament by inserting a record into the Register table.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace procedure register_team_tournament(
   p_team_id in number,
   p_tourn_id in number
) is
begin
   insert into register (teamId, tournamentId)
   values (p_team_id, p_tourn_id);
   dbms_output.put_line('Team ' || p_team_id || ' registered successfully for Tournament ' || p_tourn_id);
end;
/

begin
   register_team_tournament(305, 202);
end;
/
```



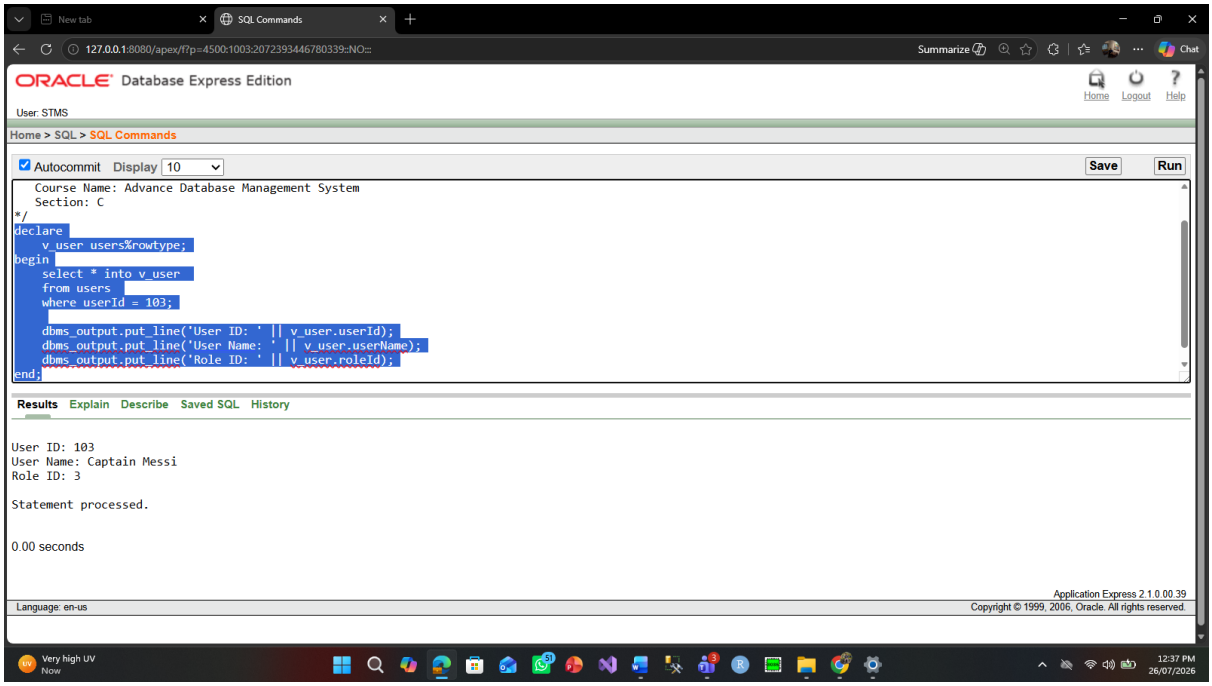
Question 5:

Write a PL/SQL program using a table-based record (%rowtype) to retrieve and display full details of a specific user.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_user users%rowtype;
begin
   select * into v_user
   from users
   where userId = 103;

   dbms_output.put_line('User ID: ' || v_user.userId);
   dbms_output.put_line('User Name: ' || v_user.userName);
   dbms_output.put_line('Role ID: ' || v_user.roleId);
end;
```

/

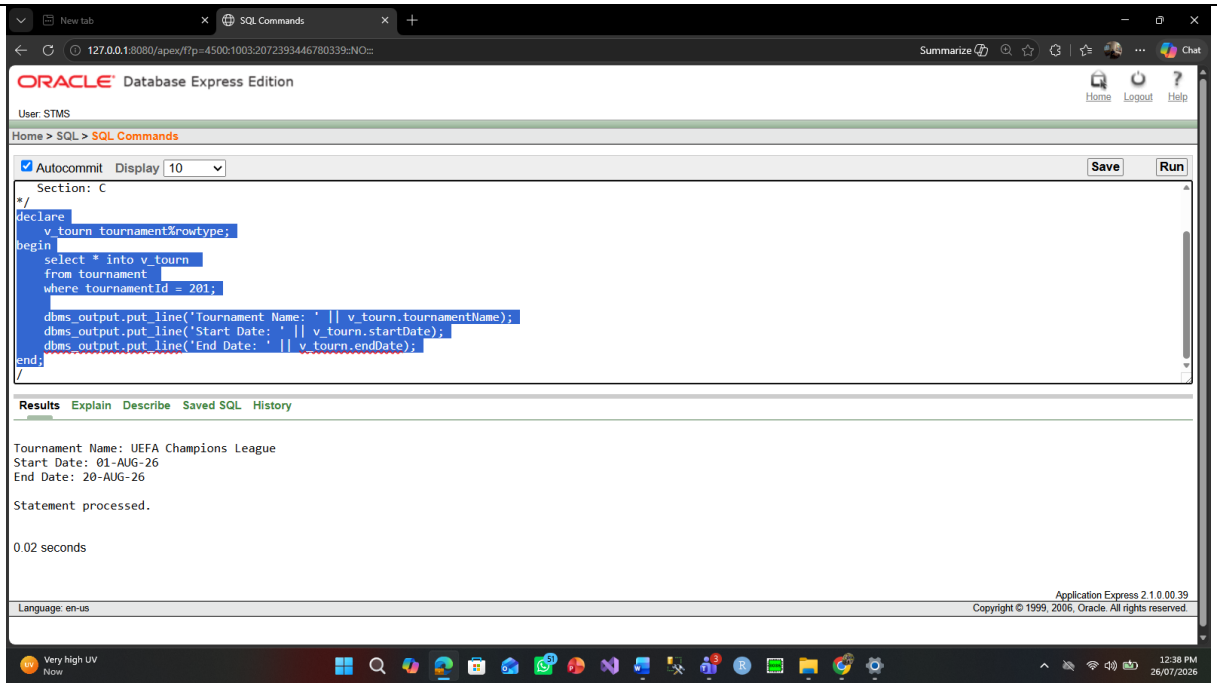


Question 6:

Write a PL/SQL program using a table-based record (%rowtype) to fetch and display information for a tournament.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   v_tourn tournament%rowtype;
begin
   select * into v_tourn
   from tournament
   where tournamentId = 201;

   dbms_output.put_line('Tournament Name: ' || v_tourn.tournamentName);
   dbms_output.put_line('Start Date: ' || v_tourn.startDate);
   dbms_output.put_line('End Date: ' || v_tourn.endDate);
end;
/
```

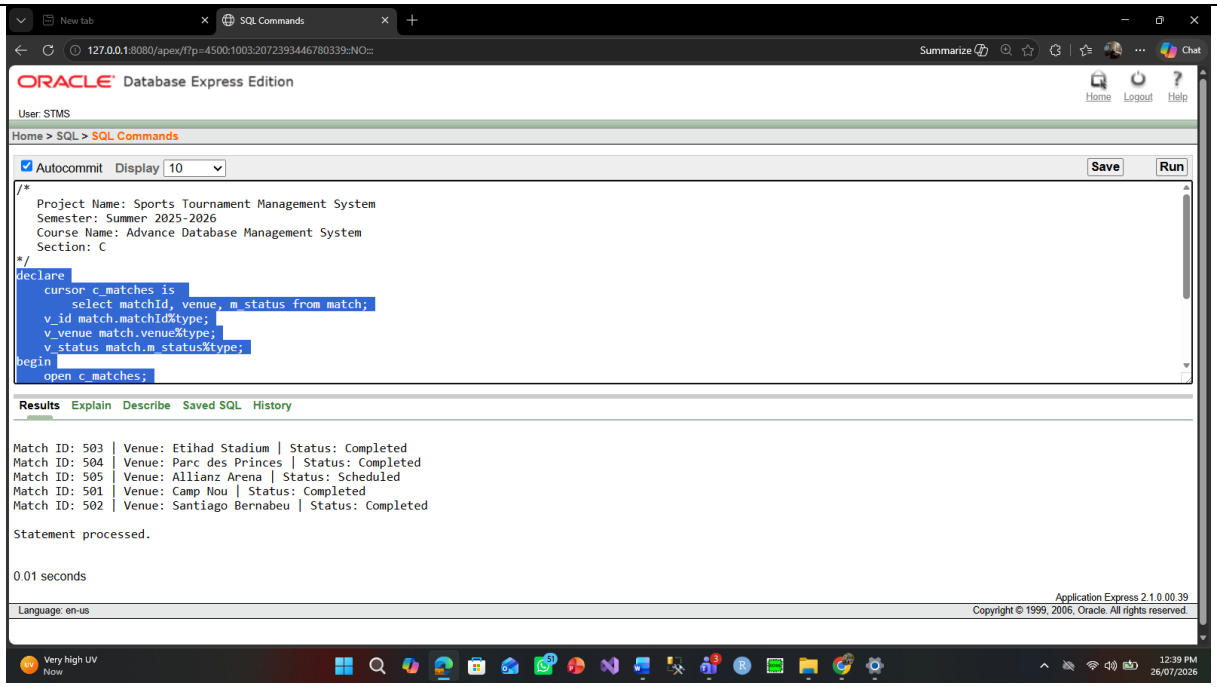


Question 7:

Write a PL/SQL block using an Explicit Cursor to iterate through all matches and print their ID, venue, and status.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   cursor c_matches is
      select matchId, venue, m_status from match;
   v_id match.matchId%type;
   v_venue match.venue%type;
   v_status match.m_status%type;
begin
   open c_matches;
   loop
      fetch c_matches into v_id, v_venue, v_status;
      exit when c_matches%notfound;
      dbms_output.put_line('Match ID: ' || v_id || ' | Venue: ' || v_venue || ' | Status: ' || v_status);
   end loop;
   close c_matches;
end;
/
```

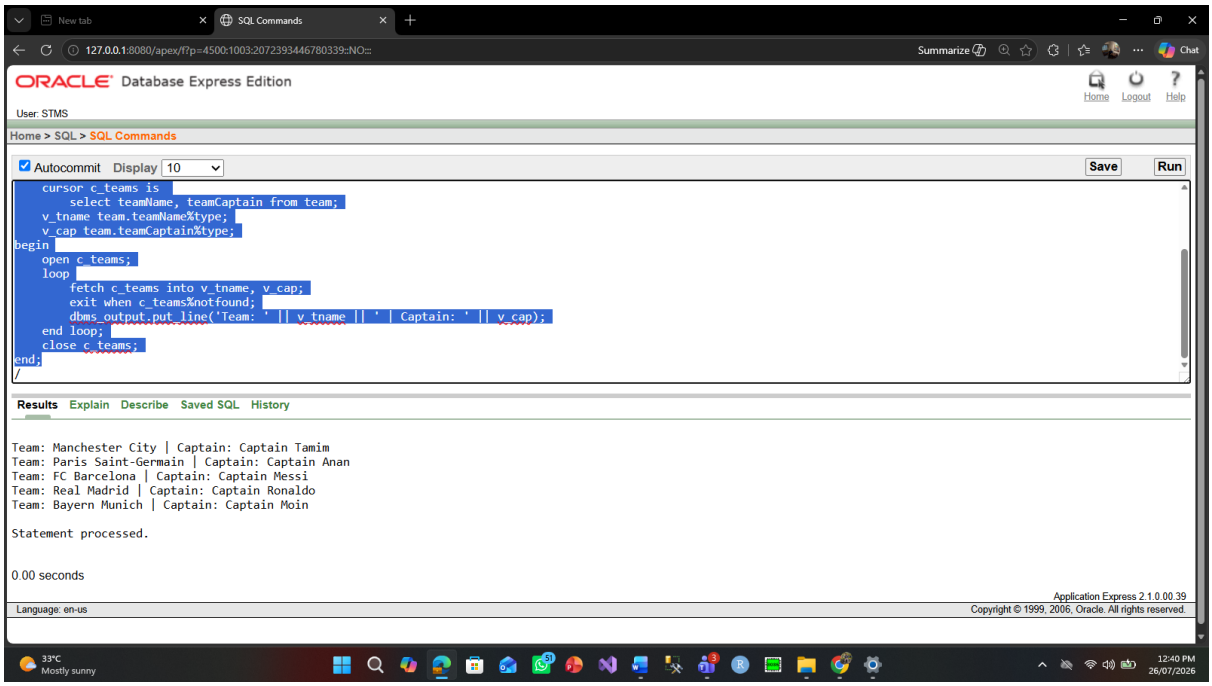


Question 8:

Write a PL/SQL block using an Explicit Cursor to display the list of all teams along with their respective captain names.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   cursor c_teams is
      select teamName, teamCaptain from team;
   v_tname team.teamName%type;
   v_cap team.teamCaptain%type;
begin
   open c_teams;
   loop
      fetch c_teams into v_tname, v_cap;
      exit when c_teams%notfound;
      dbms_output.put_line('Team: ' || v_tname || ' | Captain: ' || v_cap);
   end loop;
   close c_teams;
end;
/
```

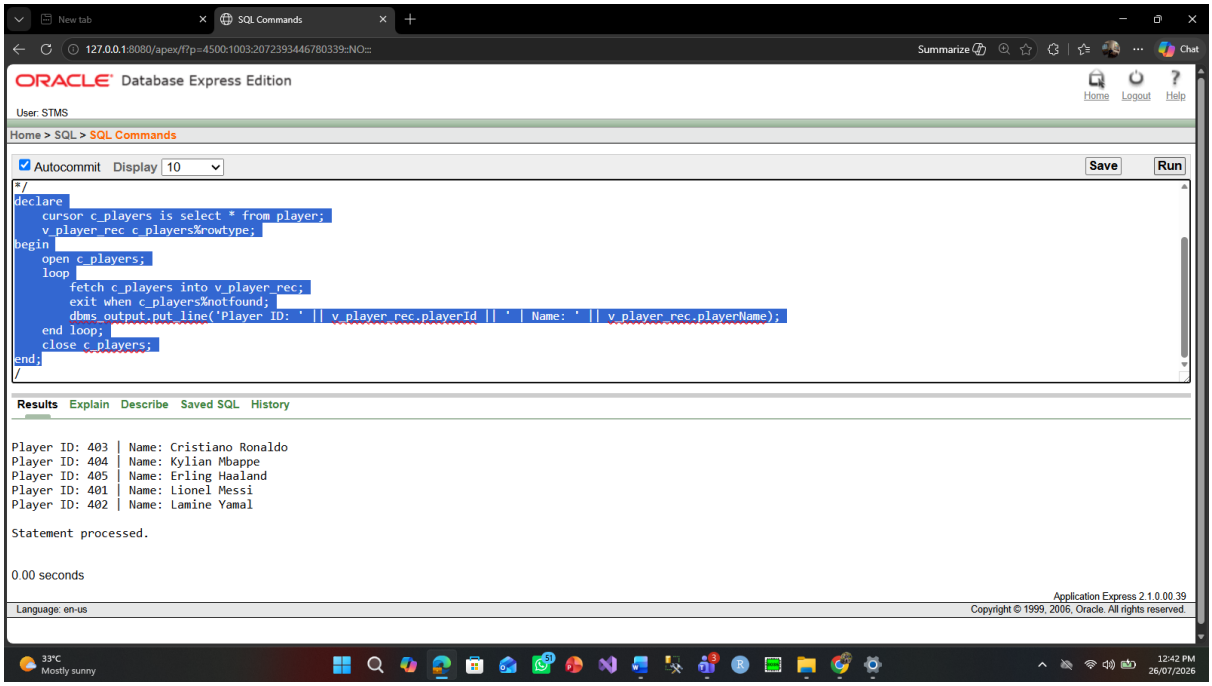


Question 9:

Write a PL/SQL program combining an Explicit Cursor with a cursor-based record (c_players%rowtype) to display player details.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   cursor c_players is select * from player;
   v_player_rec c_players%rowtype;
begin
   open c_players;
   loop
      fetch c_players into v_player_rec;
      exit when c_players%notfound;
      dbms_output.put_line('Player ID: ' || v_player_rec.playerId || ' | Name: ' || v_player_rec.playerName);
   end loop;
   close c_players;
end;
/
```

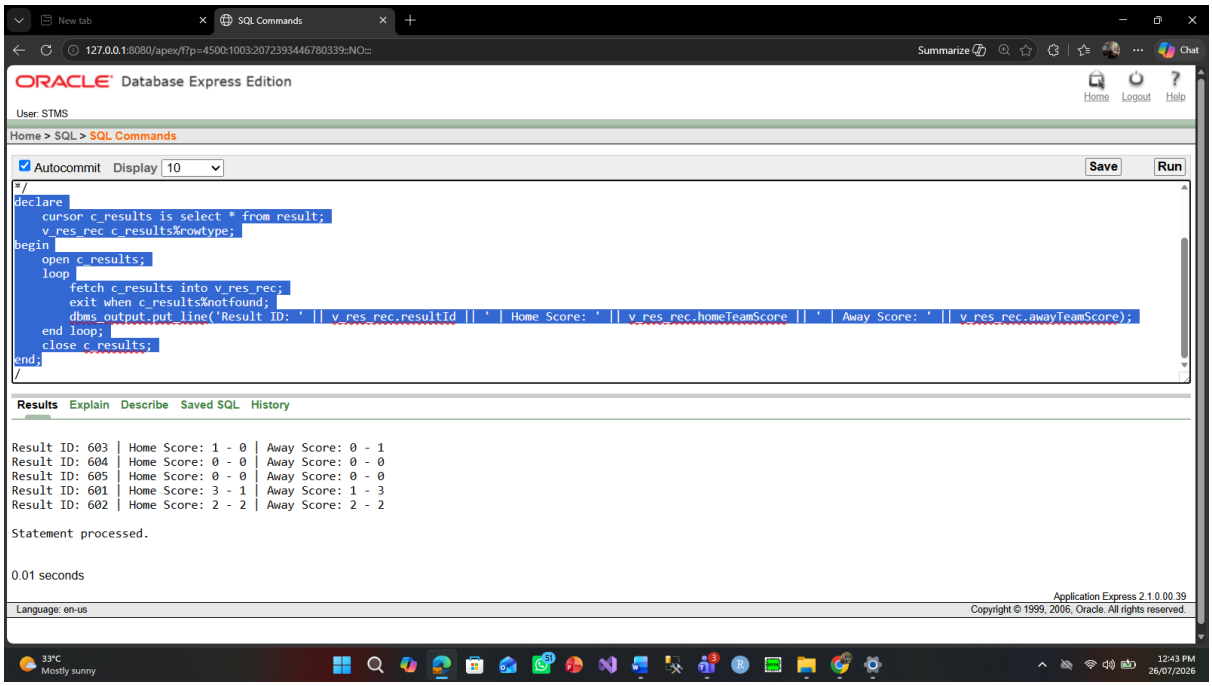


Question 10:

Write a PL/SQL program using a cursor-based record (c_results%rowtype) to display all match results and scores.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
declare
   cursor c_results is select * from result;
   v_res_rec c_results%rowtype;
begin
   open c_results;
   loop
      fetch c_results into v_res_rec;
      exit when c_results%notfound;
      dbms_output.put_line('Result ID: ' || v_res_rec.resultId || ' | Home Score: ' || v_res_rec.homeTeamScore
|| ' | Away Score: ' || v_res_rec.awayTeamScore);
   end loop;
   close c_results;
end;
/
```

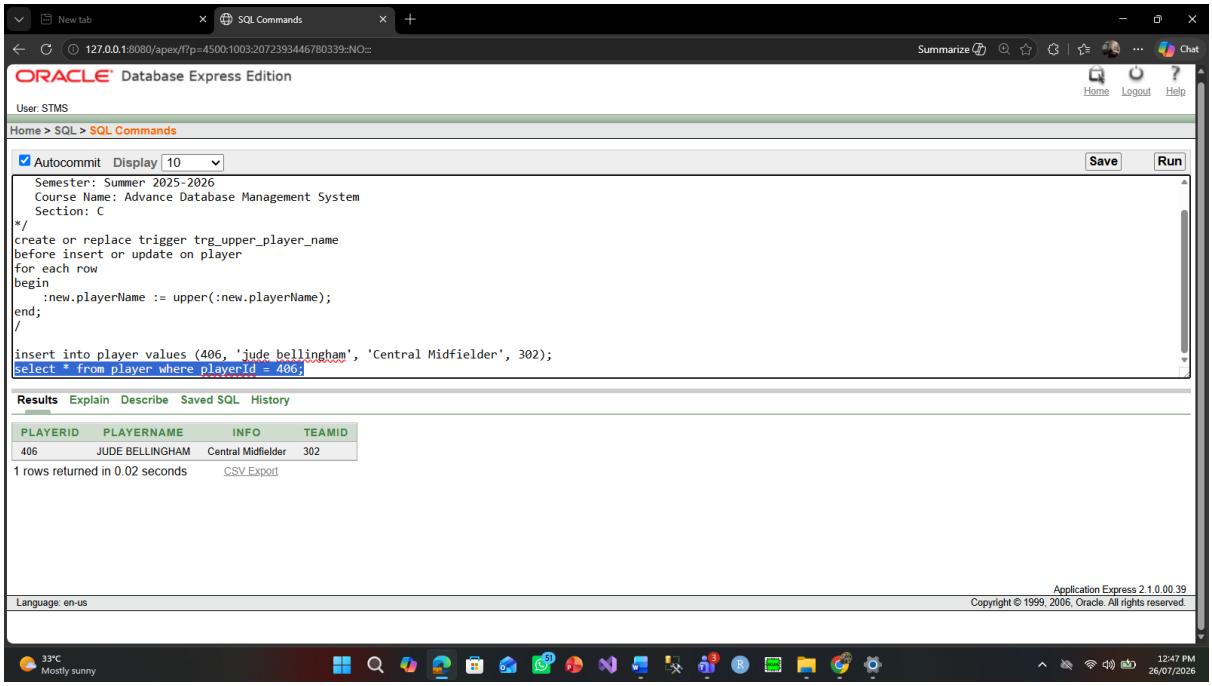


Question 11:

Create a Row-Level Trigger that automatically converts the playerName to UPPERCASE before inserting or updating records in the Player table.

Code:

```
/*
   Project Name: Sports Tournament Management System
   Semester: Summer 2025-2026
   Course Name: Advance Database Management System
   Section: C
*/
create or replace trigger trg_upper_player_name
before insert or update on player
for each row
begin
    :new.playerName := upper(:new.playerName);
end;
/

insert into player values (406, 'jude bellingham', 'Central Midfielder', 302);
select * from player where playerId = 406;
```

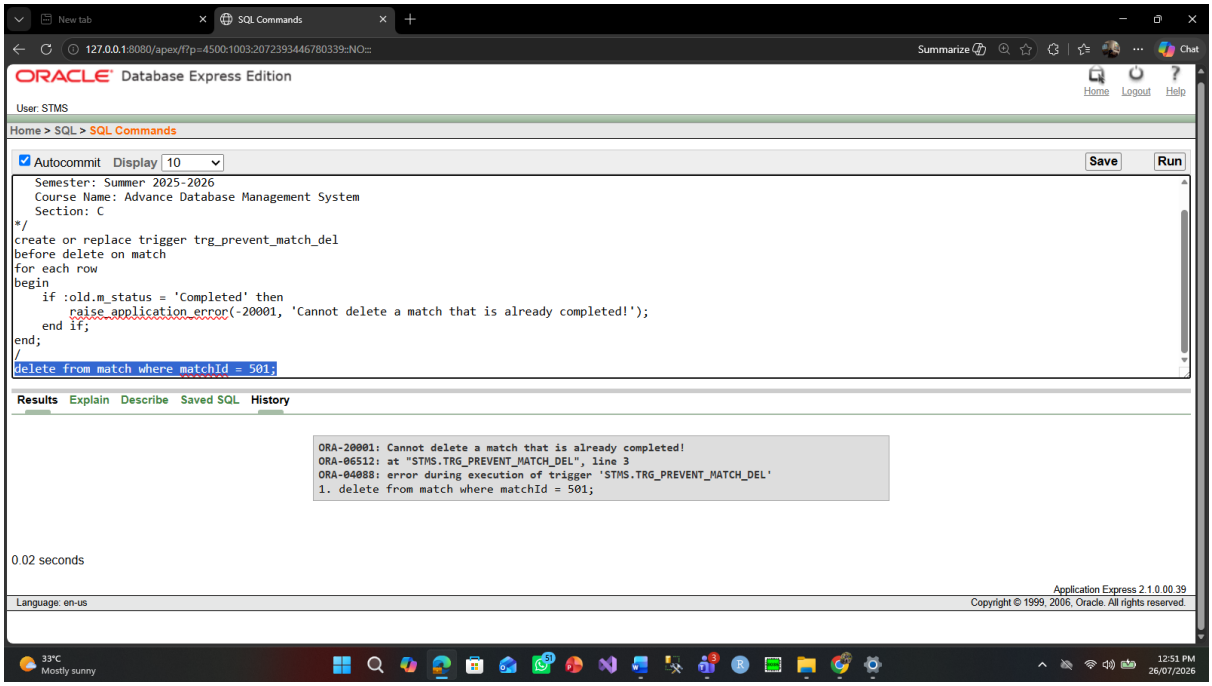


Question 12:

Create a Row-Level Trigger to prevent deletion of any match whose status is marked as 'Completed'.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace trigger trg_prevent_match_del
before delete on match
for each row
begin
   if :old.m_status = 'Completed' then
      raise_application_error(-20001, 'Cannot delete a match that is already completed!');
   end if;
end;
/
delete from match where matchId = 501;
```

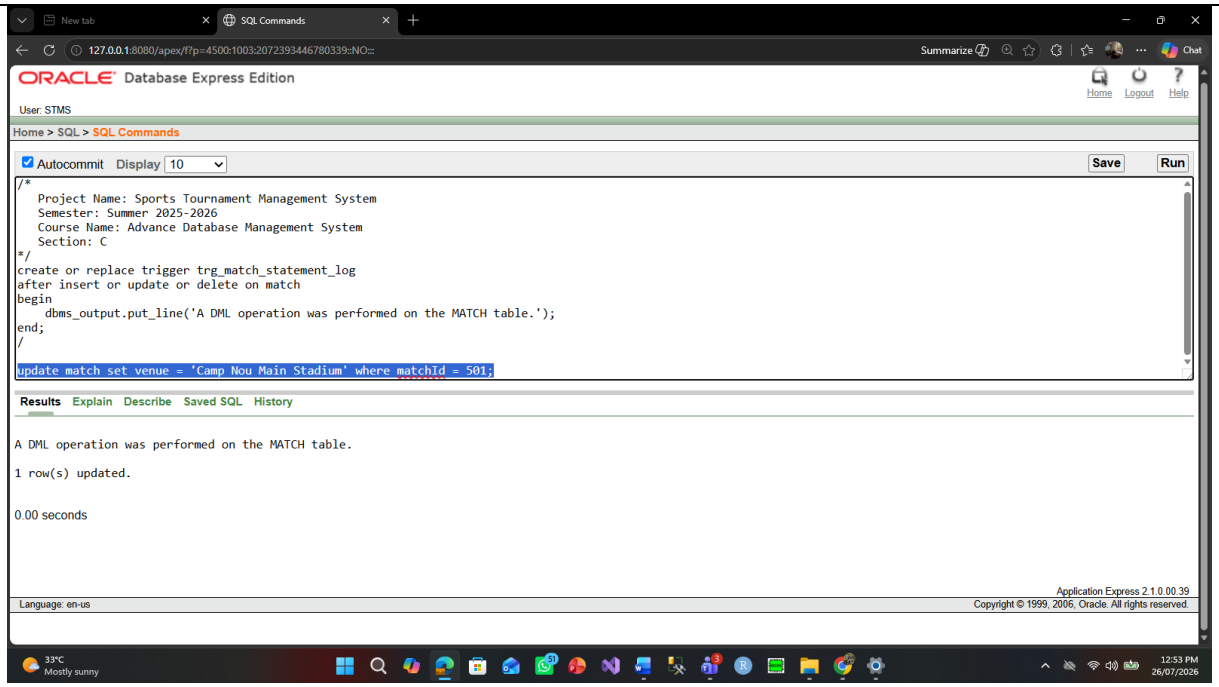


Question 13:

Create a Statement-Level Trigger that prints a notification whenever an INSERT, UPDATE, or DELETE operation occurs on the Match table.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace trigger trg_match_statement_log
after insert or update or delete on match
begin
   dbms_output.put_line('A DML operation was performed on the MATCH table.');
end;
/
update match set venue = 'Camp Nou Main Stadium' where matchId = 501;
```

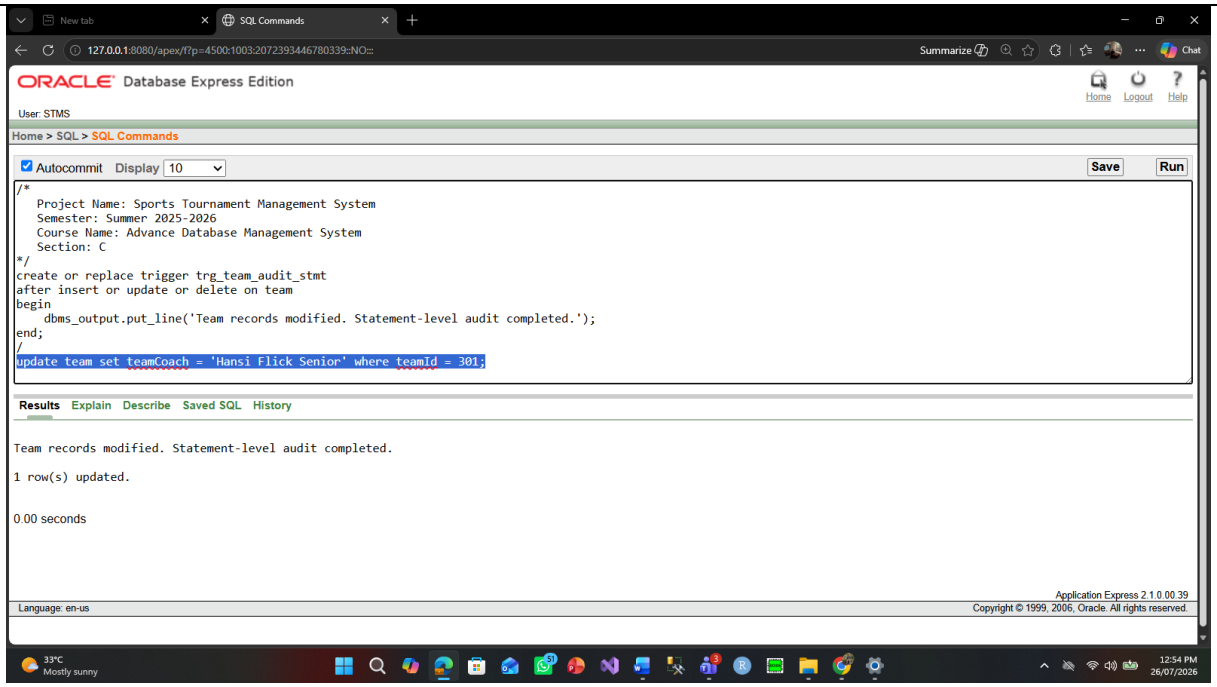


Question 14:

Create a Statement-Level Trigger that logs an audit message whenever modifications are made to the Team table.

Code:

```
/*
   Project Name: Sports Tournament Management System
   Semester: Summer 2025-2026
   Course Name: Advance Database Management System
   Section: C
*/
create or replace trigger trg_team_audit_stmt
after insert or update or delete on team
begin
    dbms_output.put_line('Team records modified. Statement-level audit completed.');
end;
/
update team set teamCoach = 'Hansi Flick Senior' where teamId = 301;
```



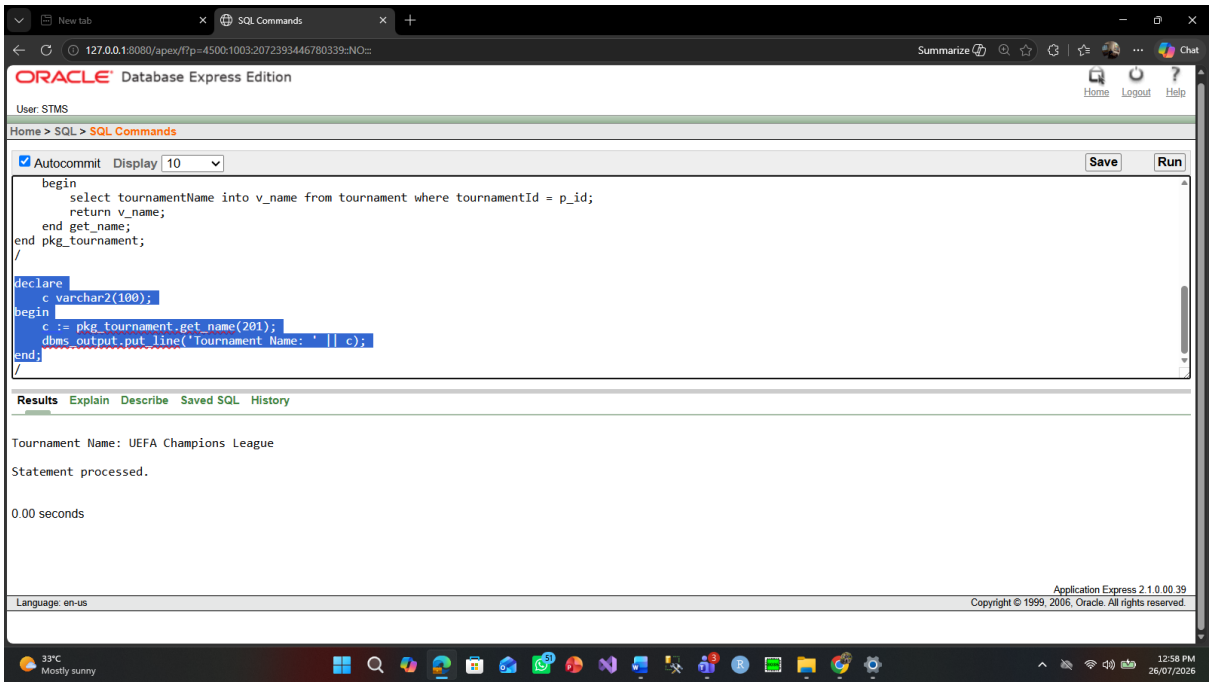
Question 15:

Create a Package (pkg_tournament) consisting of a procedure to add a new tournament and a function to retrieve tournament names.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace package pkg_tournament as
   procedure add_tournament(p_id number, p_name varchar2, p_start date, p_end date);
   function get_name(p_id number) return varchar2;
end pkg_tournament;
/

create or replace package body pkg_tournament as
   procedure add_tournament(p_id number, p_name varchar2, p_start date, p_end date) is
   begin
      insert into tournament values (p_id, p_name, p_start, p_end);
   end add_tournament;

   function get_name(p_id number) return varchar2 is
      v_name varchar2(100);
   begin
      select tournamentName into v_name from tournament where tournamentId = p_id;
      return v_name;
   end get_name;
end pkg_tournament;
/
```

```
declare
   c varchar2(100);
begin
   c := pkg_tournament.get_name(201);
   dbms_output.put_line('Tournament Name: ' || c);
end;
/
```



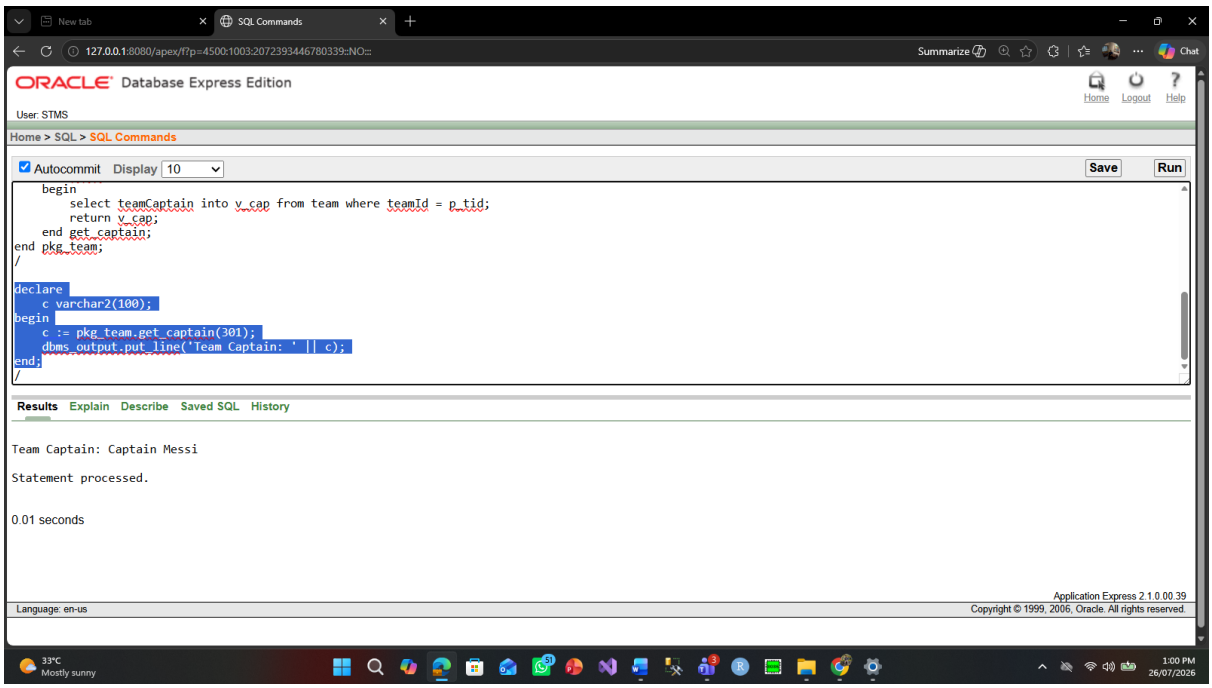
Question 16:

Create a Package (pkg_team) containing subprograms for adding a new player and fetching a team captain's name.

Code:

```
/*
  Project Name: Sports Tournament Management System
  Semester: Summer 2025-2026
  Course Name: Advance Database Management System
  Section: C
*/
create or replace package pkg_team as
   procedure add_player(p_id number, p_name varchar2, p_info varchar2, p_tid number);
   function get_captain(p_tid number) return varchar2;
end pkg_team;
/

create or replace package body pkg_team as
   procedure add_player(p_id number, p_name varchar2, p_info varchar2, p_tid number) is
   begin
      insert into player values (p_id, p_name, p_info, p_tid);
   end add_player;

   function get_captain(p_tid number) return varchar2 is
      v_cap varchar2(100);
```

```
    begin
        select teamCaptain into v_cap from team where teamId = p_tid;
        return v_cap;
    end get_captain;
end pkg_team;
/

declare
    c varchar2(100);
begin
    c := pkg_team.get_captain(301);
    dbms_output.put_line('Team Captain: ' || c);
end;
/
```



Conclusion:

The Sports Tournament Management System is a perfect example of how a database-oriented application can be used for the effective organization of sports tournaments. The presented system enables the centralization of the process of managing users, tournaments, teams, players, matches, and results. When using the proposed solution, a large number of tasks are completed automatically, which leads to the reduction of paperwork, minimizing the influence of human errors, and achieving uniformity in data. Overall, the main goal of this project is successfully achieved.

Future Work

Although the proposed system fulfills the core requirements of tournament management, several enhancements can be incorporated in the future to improve its functionality and user experience. Possible future improvements include:

- Developing a web-based or mobile application for easier accessibility.
- Implementing secure user authentication with encrypted passwords.
- Adding online team registration and tournament registration features.
- Integrating real-time match score updates and live notifications.
- Generating analytical reports and tournament statistics automatically.
- Supporting multiple sports with customizable tournament formats.
- Providing cloud-based database support for better scalability and data backup.

With these enhancements, the system can become a more comprehensive, scalable, and practical solution for managing sports tournaments in real-world environments.